BOSTON UNIVERSITY

EC 551 – Advanced Digital Design with Verilog and FPGAs

Lecture 11
Timing Analysis and Correction

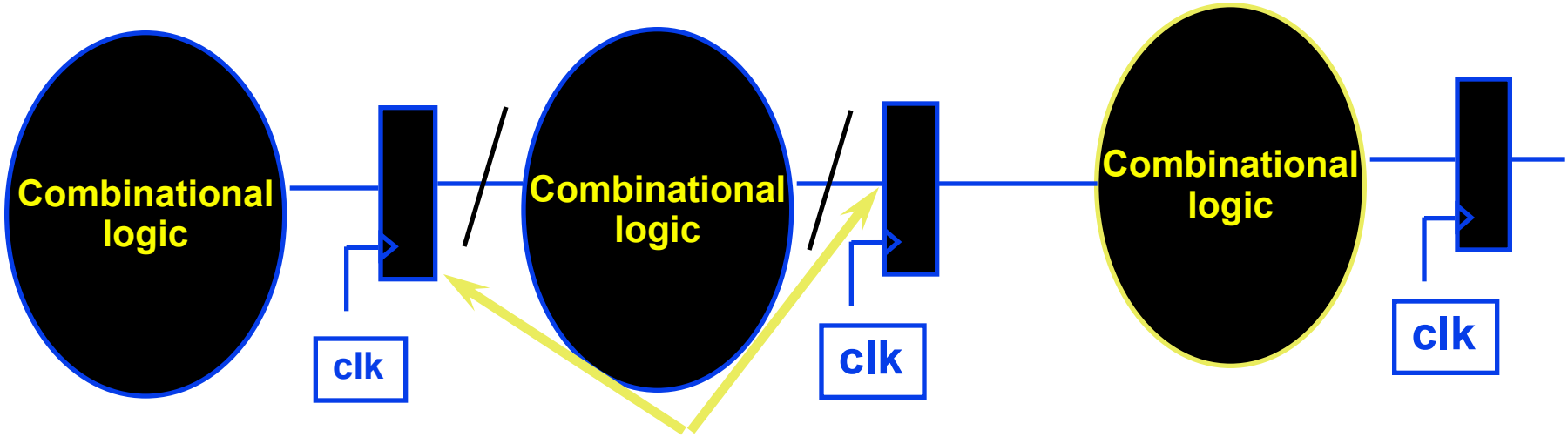
Prof. Douglas Densmore

Additional material courtesy of Alexander Taubin, EC551 Fall 2009
ECE 260B – CSE 241A Timing Analysis & Correction
http://vlsicad.ucsd.edu/courses/ece260b-w05
http://vlsicad.ucsd.edu

Timing Analysis

- Testing
- Simulation
  - Device modeling (BSIM)
  - Transistor-level time domain analysis (SPICE)
  - Frequency domain interconnect analysis (AWE, PRIMA)
- Static timing analysis
  - Transistor-level (PathMill)
  - Gate-level (PrimeTime)

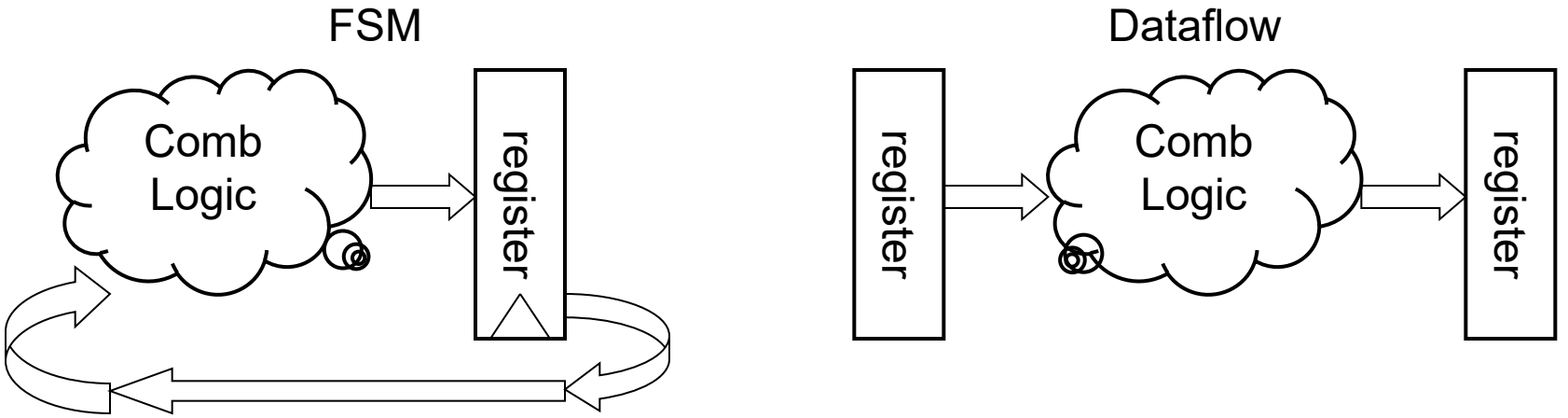
Sequential Machine



- State is stored in registers (flip-flops or latches)
- Combinational logic computes next-state, outputs from present-state, inputs

Why Clocks?

- Clocks provide the means to synchronize
  - By allowing events to happen at known timing boundaries, we can sequence these events
- Greatly simplifies building of state machines
- No need to worry about variable delay through combinational logic (CL)
  - All signals delayed until clock edge (clock imposes the worst case delay)



Courtesy K. Yang, UCLA

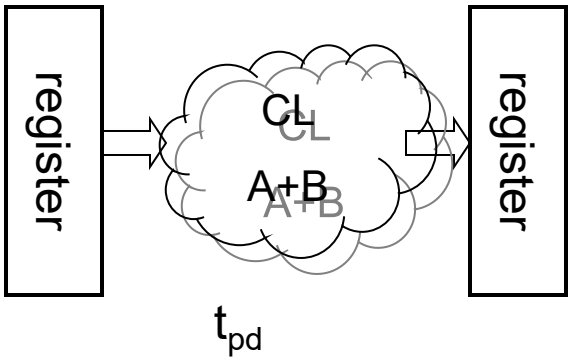
Clock Cycle Time

- Cycle time is determined by the delay through the CL
  - Signal must arrive before the latching edge
  - If too late, it waits until the next cycle
    - Synchronization and sequential order becomes incorrect
- Constraint: $T_{cycle} > T_{prop_delay_through_CL} + T_{overhead}$
  - Example: 3.0 GHz Pentium-4 → T_{cycle} = 333ps
- Can change circuit architecture to obtain smaller T_{cycle}

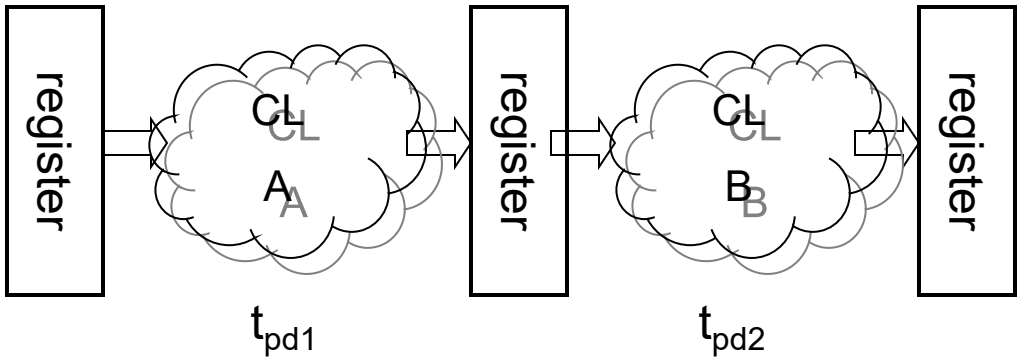
Pipelining

- For dataflow:
  - Instead of a long critical path, split the critical path into chunks
  - Insert registers to store intermediate results
  - This allows 2 waves of data to coexist within the CL
- Can we extend this ad infinitum?
  - Overhead eventually limits the pipelining
    - E.g., 1.5 to 2 gate delays for latch or FF
  - Granularity limits as well
    - Minimum time quantum: delay of a gate

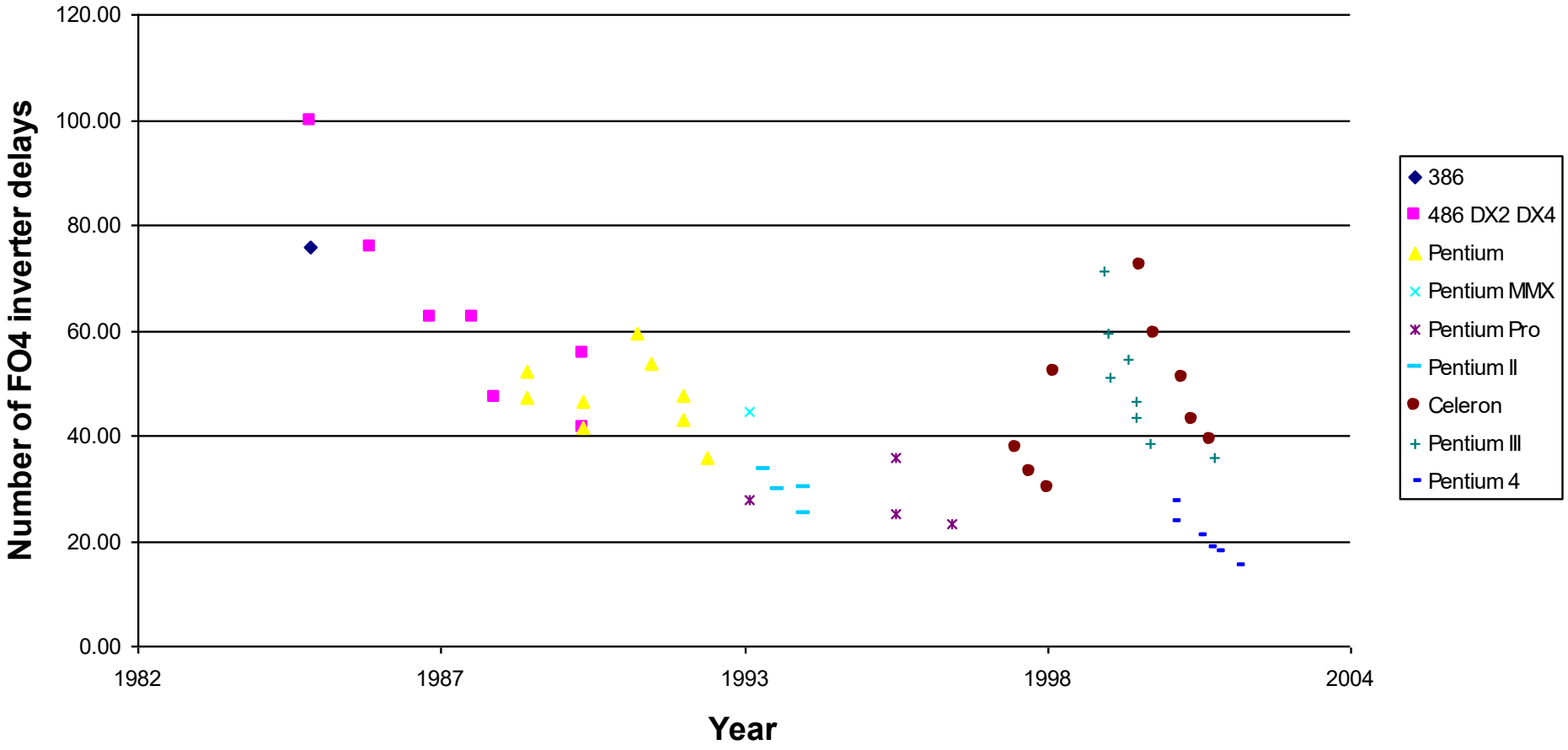
- $T_{cycle} > T_{pd} + T_{overhead}$



- $T_{cycle} > \max(t_{pd1}, t_{pd2}) + T_{overhead}$



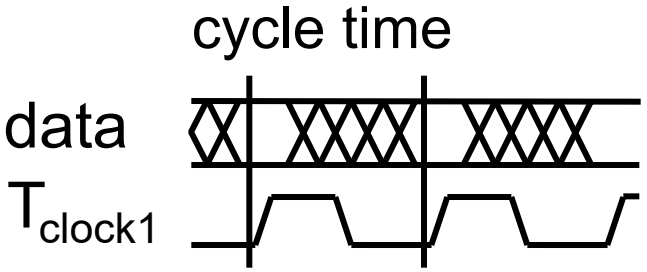
Intel MPU FO4 INV Delays Per Clock Period



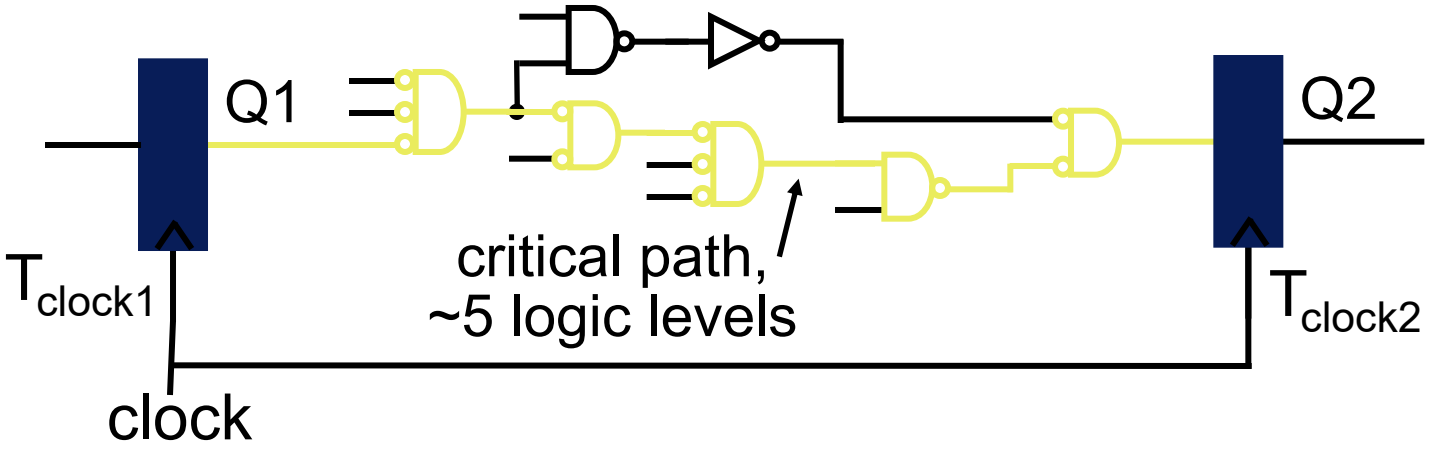
- FO4 INV = inverter driving 4 identical inverters (no interconnect)
- Half of frequency improvement has been from reduced logic stages, i.e., pipelining

Let's Revisit Cycle Time and Path Delay

- Cycle time (T) cannot be smaller than longest path delay (T_{max})
- Longest (critical) path delay is a function of:
- Total gate, wire delays
  - logic levels



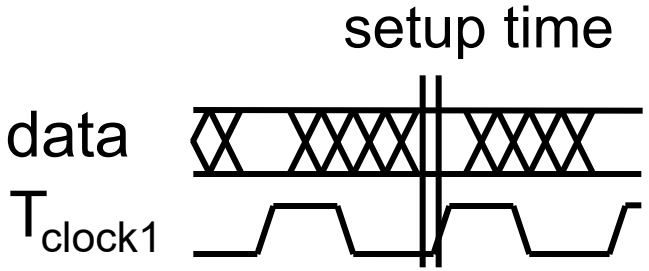
$T_{max} \leq T$



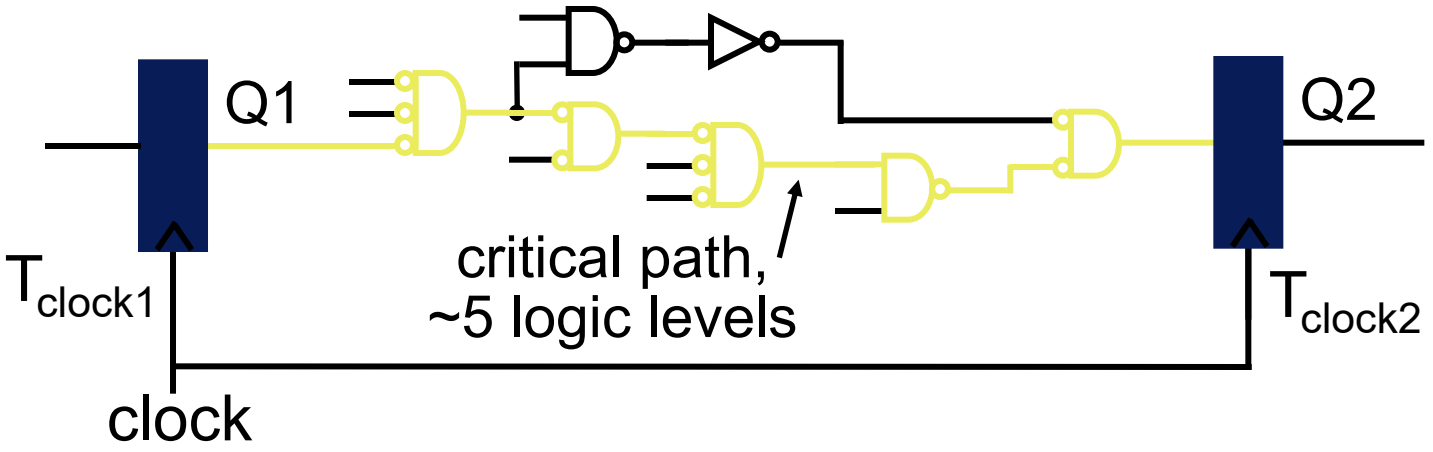
Courtesy K. Keutzer et al. UCB

Cycle Time - Setup Time

- For FFs to correctly capture data, must be stable for:
- Setup time (T_{setup}) *before* clock arrives



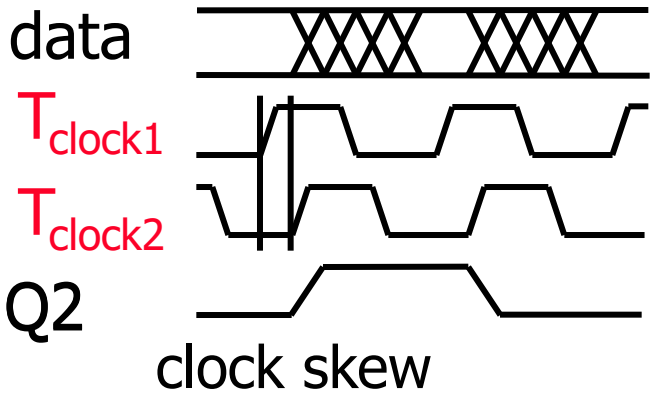
$T_{max} + T_{setup} \leq T$



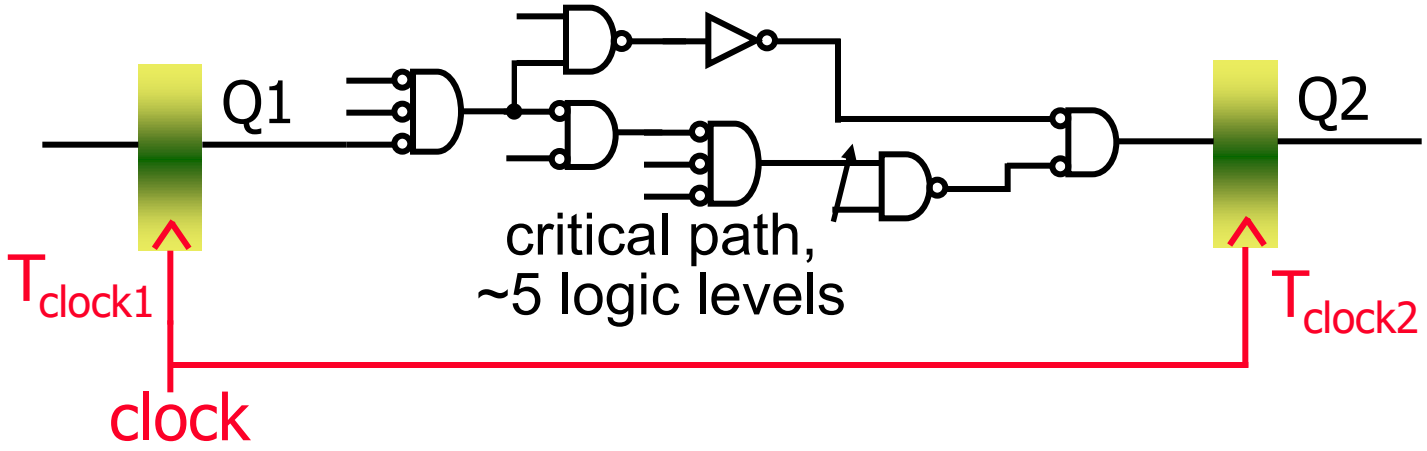
Courtesy K. Keutzer et al. UCB

Cycle Time – Clock Skew

- **If clock network has unbalanced delay – clock skew**
- **Cycle time is also a function of clock skew (T_{skew})**

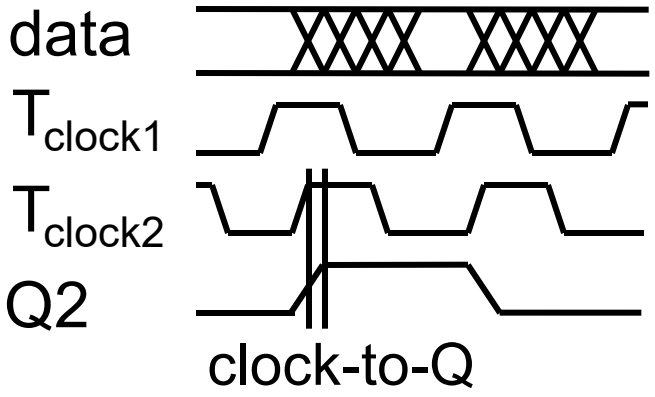


$$T_{max} + T_{setup} + T_{skew} \leq T$$



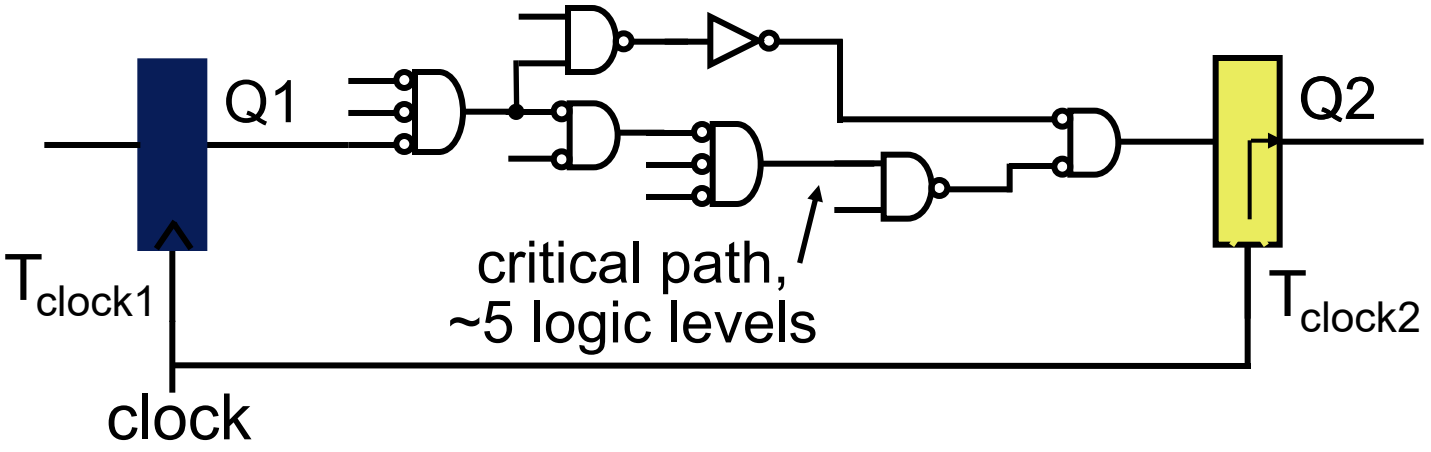
Cycle Time – Flip-Flop Delay (Clock to Q)

- Cycle time is also a function of propagation delay of FF ($T_{clk\text{-}to\text{-}Q}$ or T_{c2q})



- T_{c2q} : time from arrival of clock signal till change at FF output)

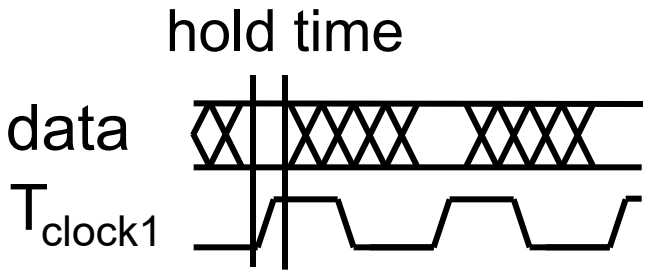
$$T_{max} + T_{setup} + T_{skew} + T_{clk-to-Q} \leq T$$



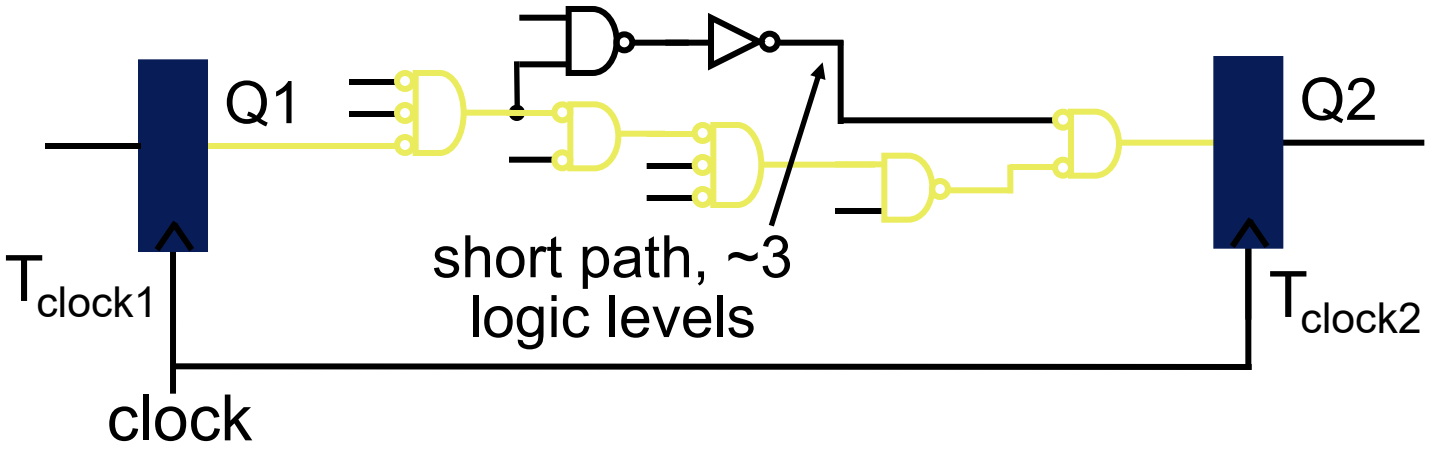
Courtesy K. Keutzer et al. UCB

Min Path Delay - Hold Time

- For FFs to correctly latch data, data must be stable during:
- Hold time (T_{hold}) *after* clock arrives
- Determined by delay of shortest path in circuit (T_{min}) and clock skew (T_{skew})

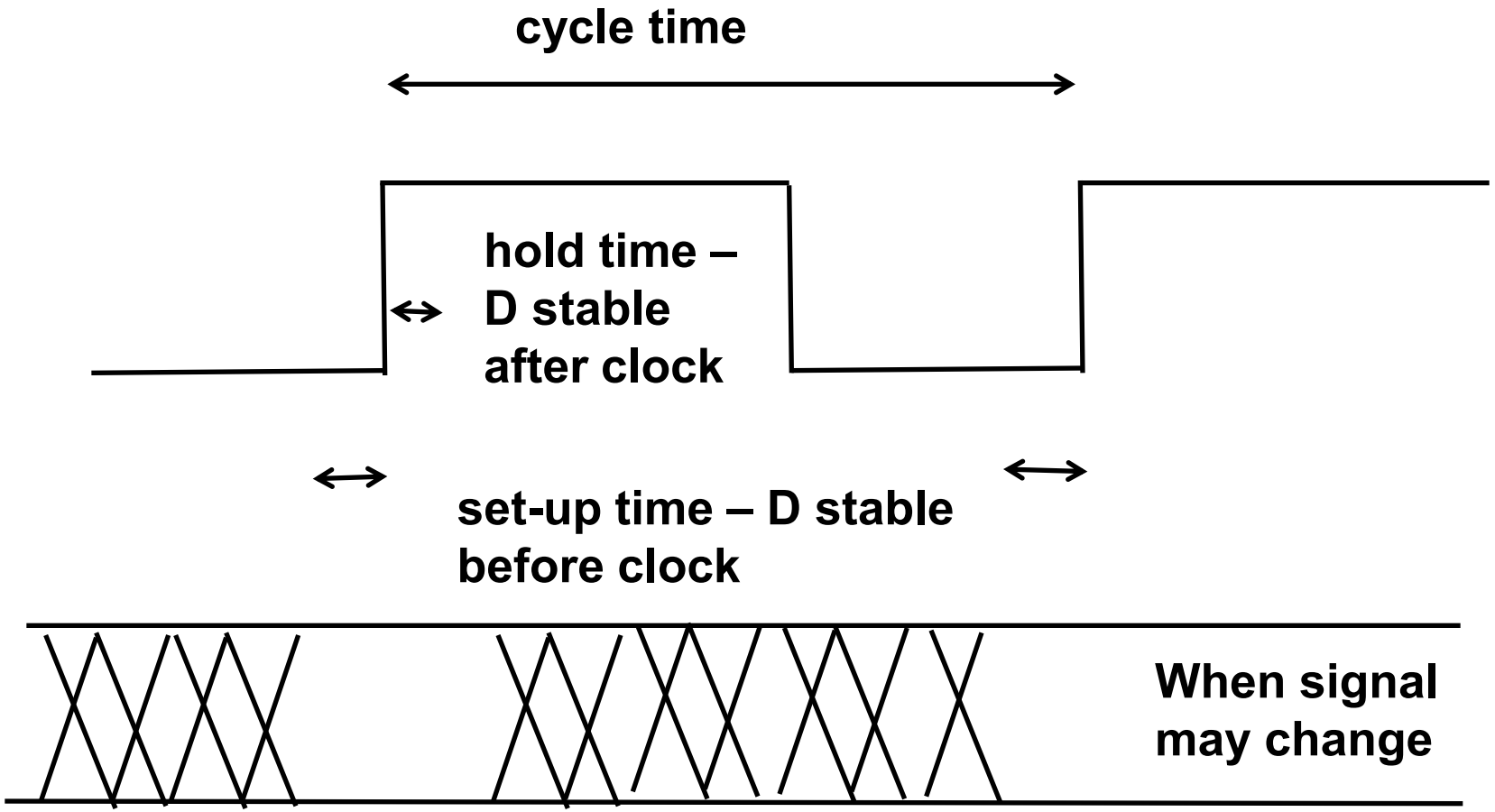


$$T_{min} \geq T_{hold} + T_{skew}$$



Courtesy K. Keutzer et al. UCB

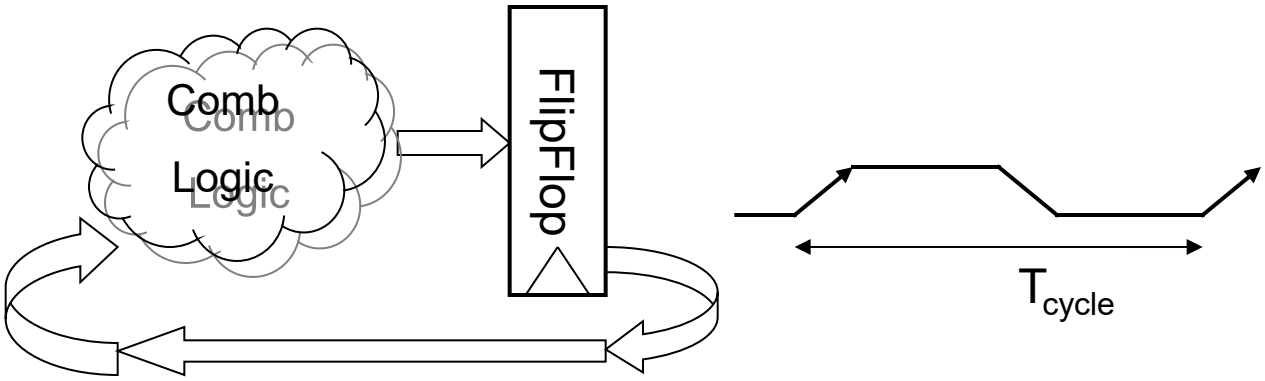
Setup, Hold, Cycle Times



Example of a single phase clock

Courtesy K. Keutzer et al. UCB

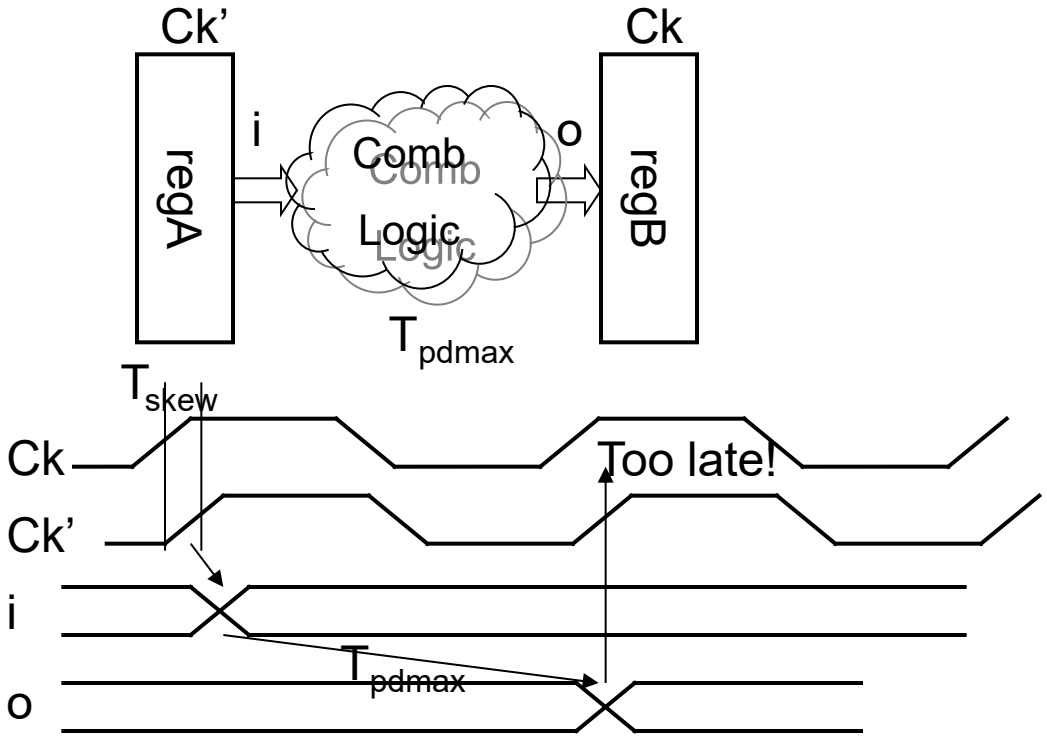
Timing Constraints for Edge-Triggered FFs



- $Max(T_{pd}) < T_{cycle} - T_{setup} - T_{c2q} - T_{skew}$
  - Delay is too long for data to be captured
- $Min(T_{pd}) > T_{hold} - T_{c2q} + T_{skew}$
  - Delay is too short and data can race through, skipping a state

Example of T_{pdmax} Violation

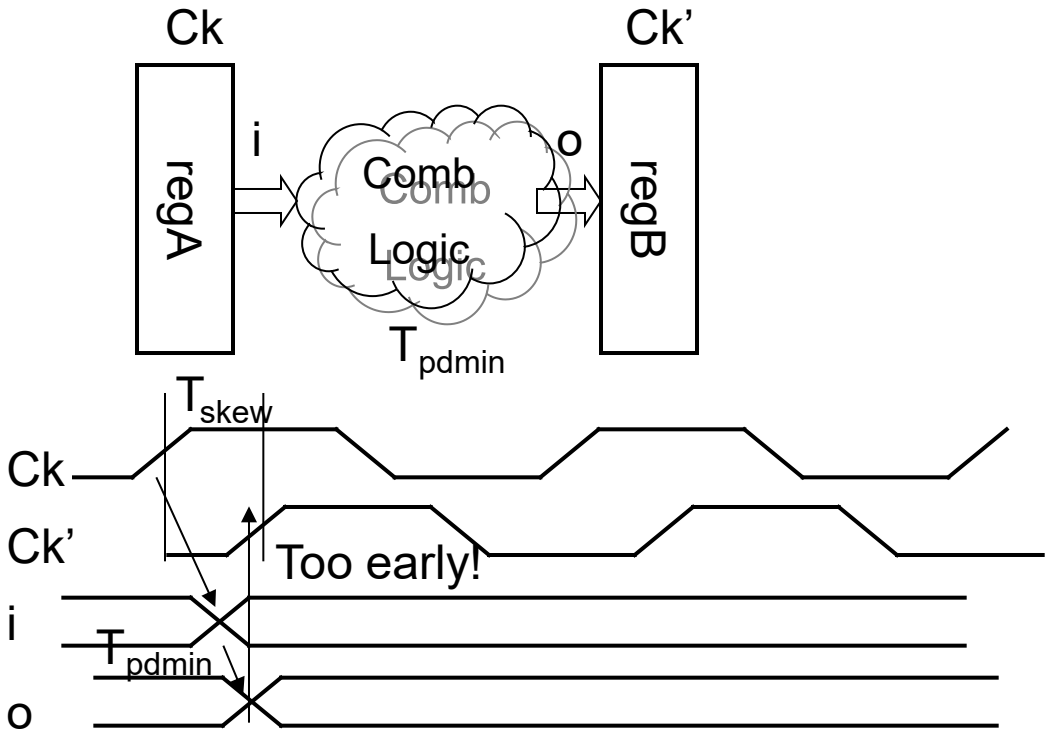
- Suppose there is skew between the registers in a dataflow (regA after regB)
- "i" gets its input values from regA at transition in Ck'
- CL output "o" arrives after Ck transition due to skew
- To correct this problem, can *increase* cycle time



Courtesy K. Yang, UCLA

Example of T_{pdmin} Violation: Race Through

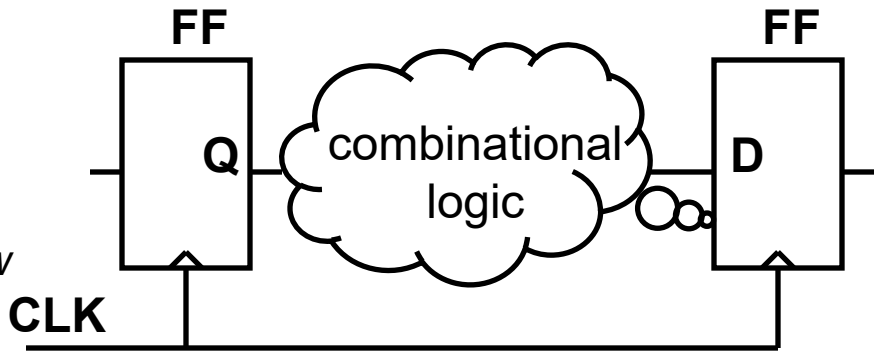
- Suppose clock skew causes regA to be clocked before regB
- "i" passes through the CL with little delay (tpdmin)
- "o" arrives before the rising Ck' causes the data to be latched
- Cannot be fixed by changing frequency → have rock instead of chip

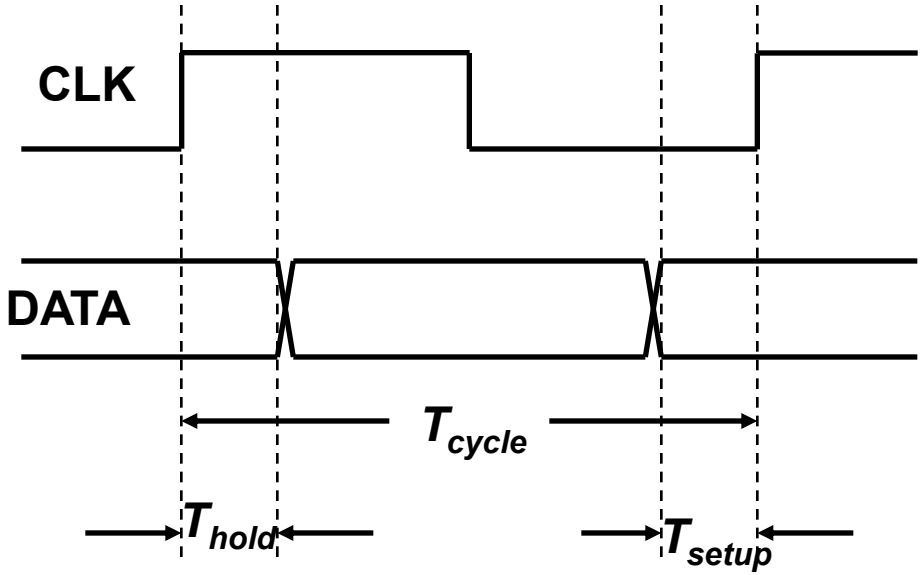


Courtesy K. Yang, UCLA

Summary: Timing Constraints

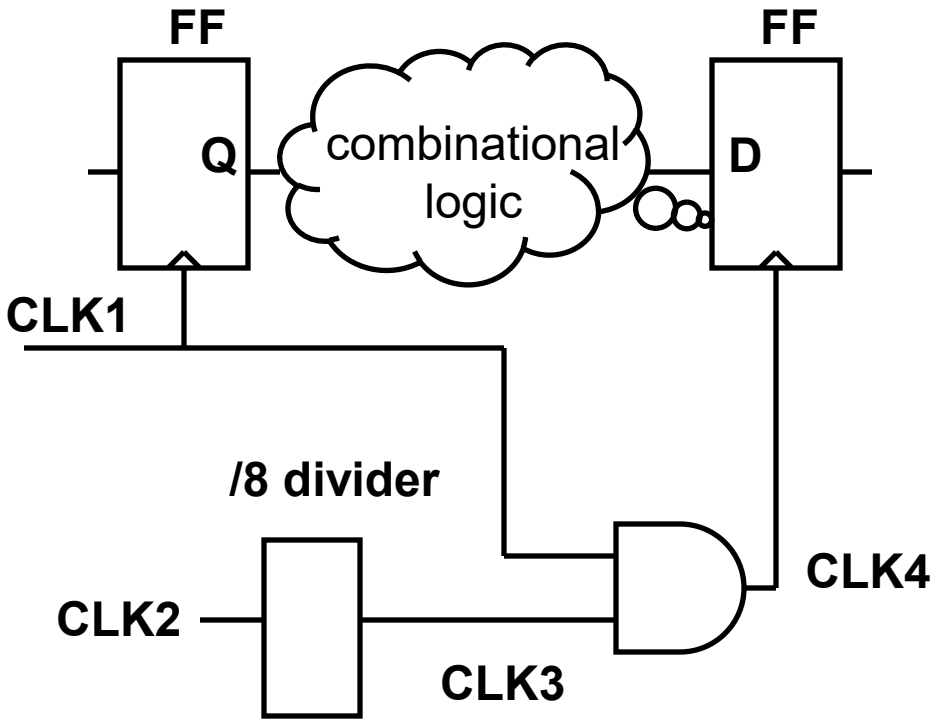
- Synchronous design = combinational logic + sequential elements
- For each flip-flop:
  - $T_{max} + T_{setup} < T_{cycle} - T_{skew}$
  - $T_{min} > T_{hold} + T_{skew}$
- T_{max} : *longest data propagation path delay*
- T_{min} : *shortest data propagation path delay*





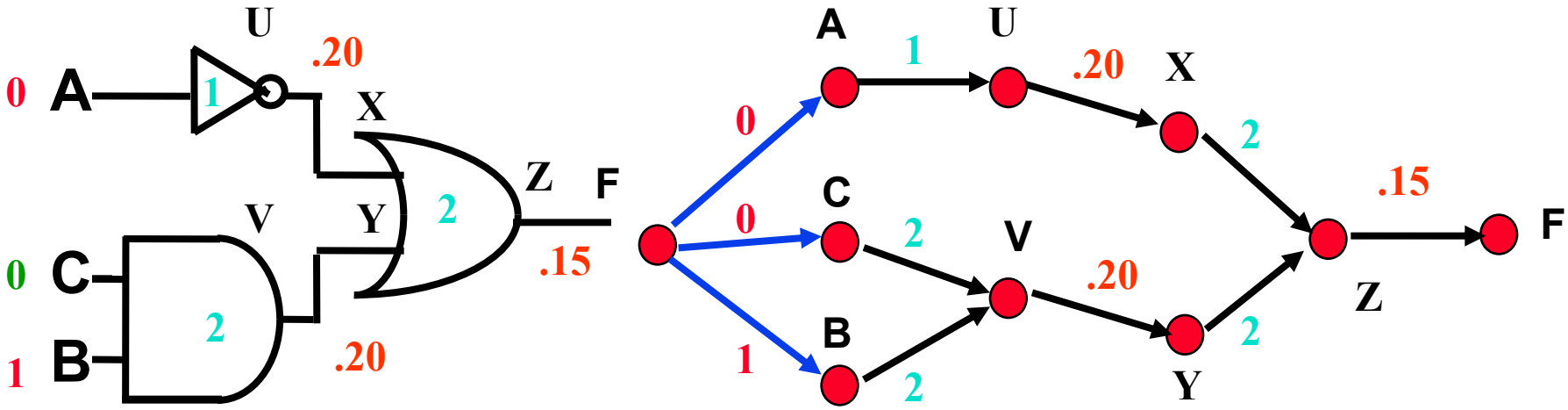
Clock Identification

- Partition the design
- Clock network
  - Clock definition
  - Derived clock
  - Clock groups
  - Clock delay (skew) calculation
  - Timing constraints exist between clocks with a common divisor frequency
- Data paths with timing constraints



Timing Graph

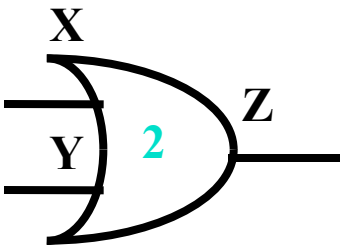
- Data paths with timing constraints
  - Starting from primary inputs/FF outputs
  - Ending at primary outputs/FF inputs
- Represented by a labeled *directed* graph $G = \langle V,E \rangle$
  - *Timing node V* ~ pin/primary input/output
  - *Timing edge E* ~ gate/wire delay
  - (Timing arc ~ gate delay)

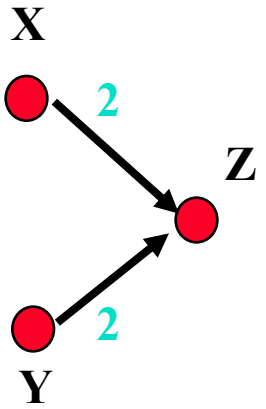


Courtesy K. Keutzer et al. UCB

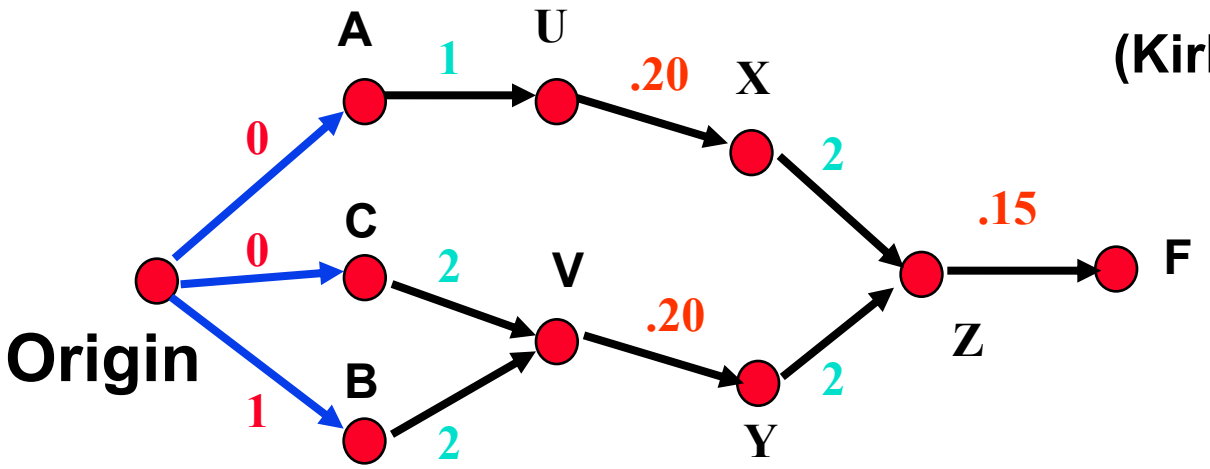
Characterization

- Static analysis = vector-less worst case analysis
  - Graph based path propagation
  - No logics
- Pre-characterized look-up tables for gate delays
  - *Min/max/rise/fall*
- Characterized interconnect delays
  - On-the-fly delay calculation
  - SDF (standard delay format) annotation





Compute Longest Path



(Kirkpatrick 1966, IBM JRD)

Compute longest path in a DAG *G = <V,E,delay,Origin>*

// delay is set of labels, Origin is the super-source of the DAG

Forward-prop(*W*){

for each vertex *v* in *W*

for each edge <*v,w*> from *v*

Final-delay(w) = max(Final-delay(w), delay(v) + delay(w) + delay(<v,w>))

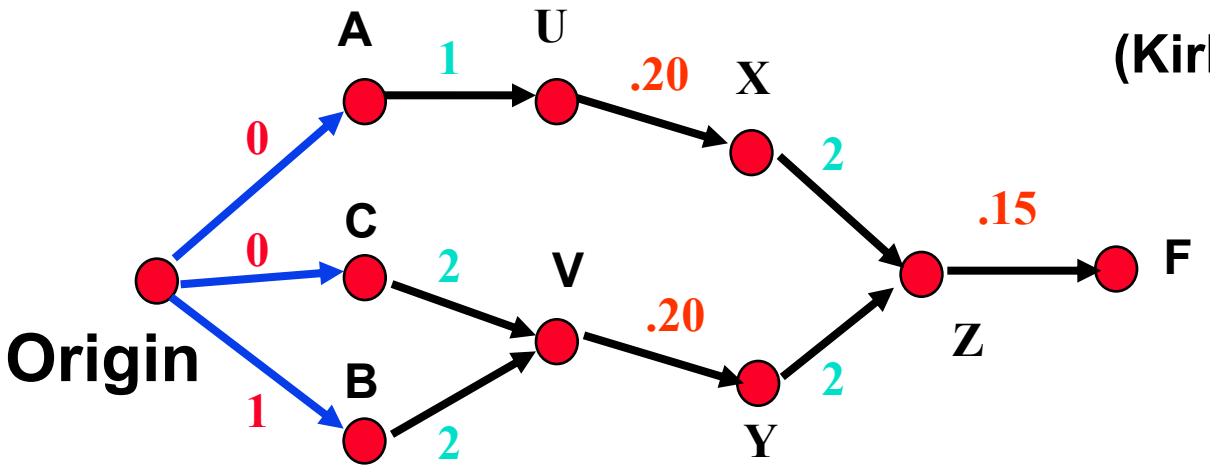
if all incoming edges of *w* have been traversed, add *w* to *W*

}

Longest path(G){

Forward_prop(Origin) }

Compute Longest Path



(Kirkpatrick 1966, IBM JRD)

Compute longest path in a DAG *G = <V,E,delay,Origin>*

// delay is set of labels, Origin is the super-source of the DAG

Forward-prop(*W*){

for each vertex *v* in *W*

for each edge *<v,w>* from *v*

Final-delay(w) = max(Final-delay(w), delay(v) + delay(w) + delay(<v,w>))

if all incoming edges of *w* have been traversed, add *w* to *W*

}

Longest path(G){

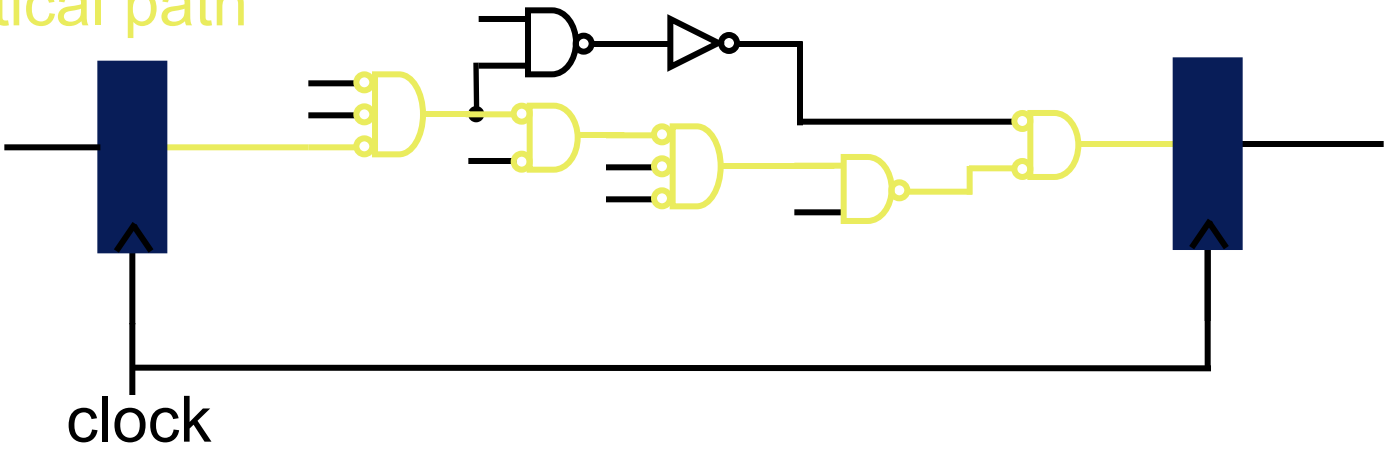
Forward_prop(Origin) }

- Dynamic programming
- *How to exclude a set of paths?*

Courtesy K. Keutzer et al. UCB

Timing Analysis Terminology

- Actual arrival time (AAT): forward propagation
- Required arrival time (RAT): backward propagation
- Slack = RAT - AAT
  - A measure of how much timing margin exists at each node
  - Slack < 0 ➔ timing violation
  - Can optimize a particular branch
  - Can trade slack for power, area, robustness
- Critical path

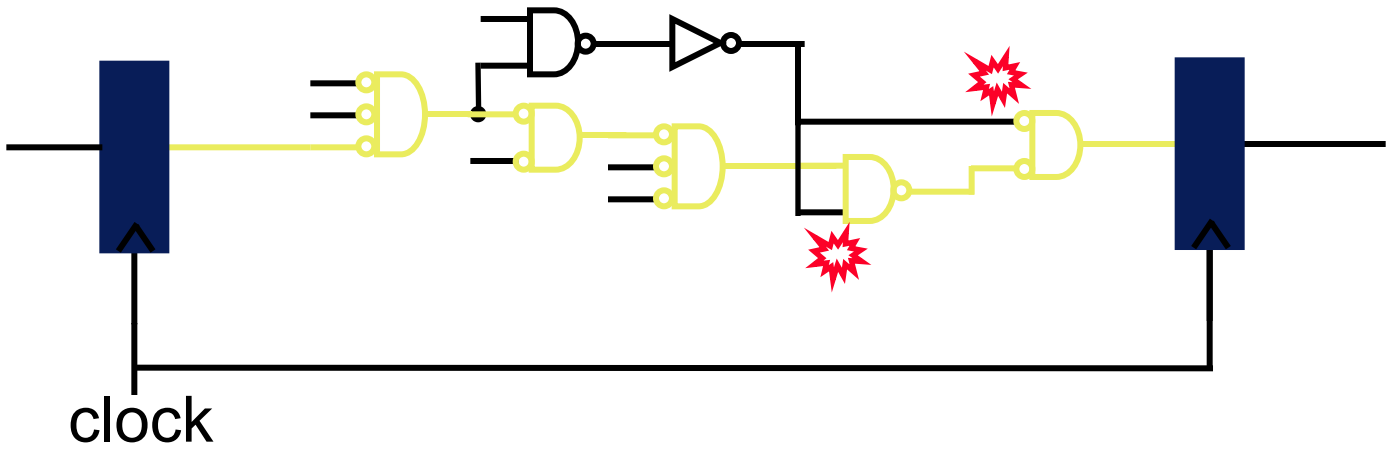


Static Timing Analysis Flow

- Read in
  - design (LEF/DEF)
  - timing library (.lib)
  - timing constraints (GCF)
  - delay annotation (SDF)
- Set up constraints
  - Annotated delays
  - IO path constraints
  - Single cycle setup/hold checks
  - Timing exceptions
    - False paths
    - Multi-cycle paths
    - Max delay constraints
    - Min delay constraints
- Construct timing graph
  - Partition clock domain (form path groups)
  - Ideal/propagated clock
  - Case analysis
- AAT propagation
  - Levelization
- Timing report
  - End points with violations
  - Path enumeration

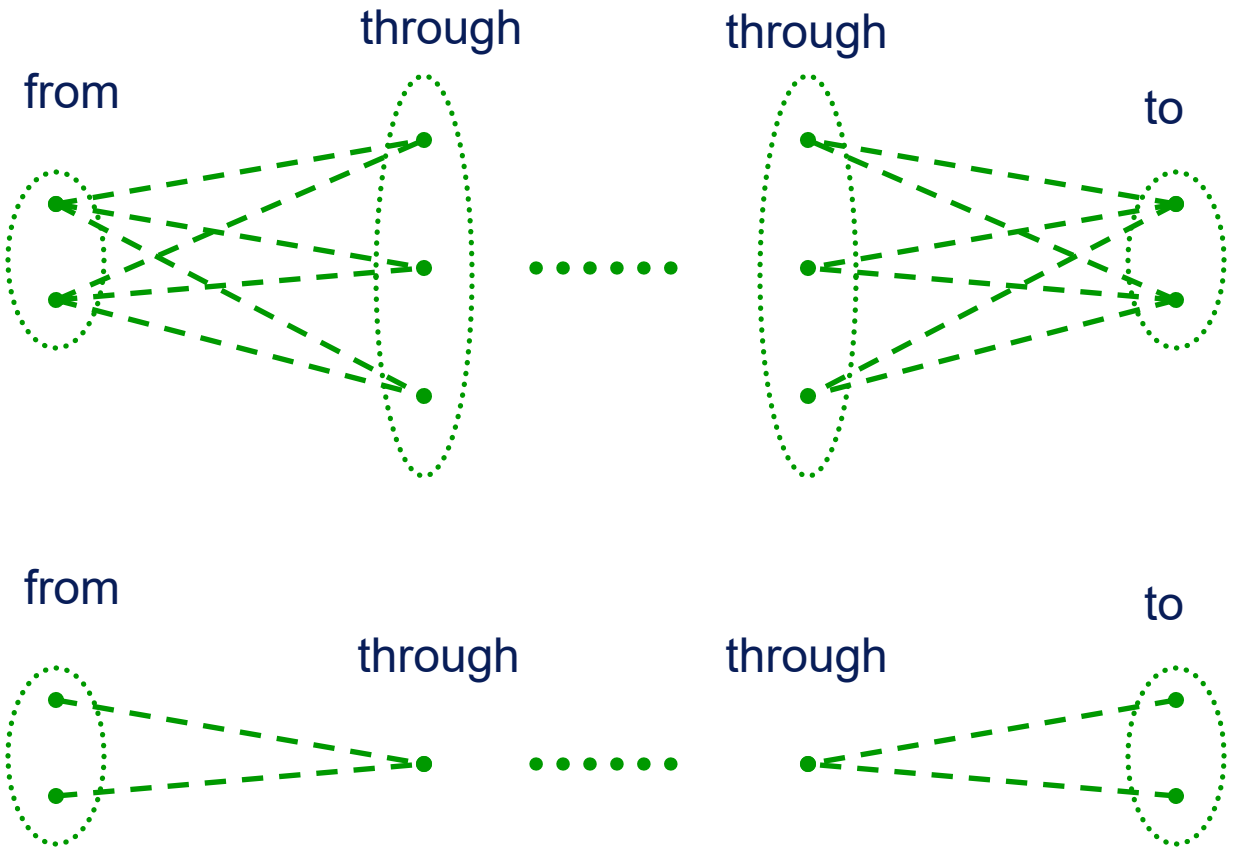
Timing Exceptions

- False paths: topologically connected but logically impossible to enable
- To enable a path
  - Logically: non-controlling values (e.g., 0 for OR gates, 1 for AND gates) at side inputs
  - Temporally: earlier signal transitions at side inputs



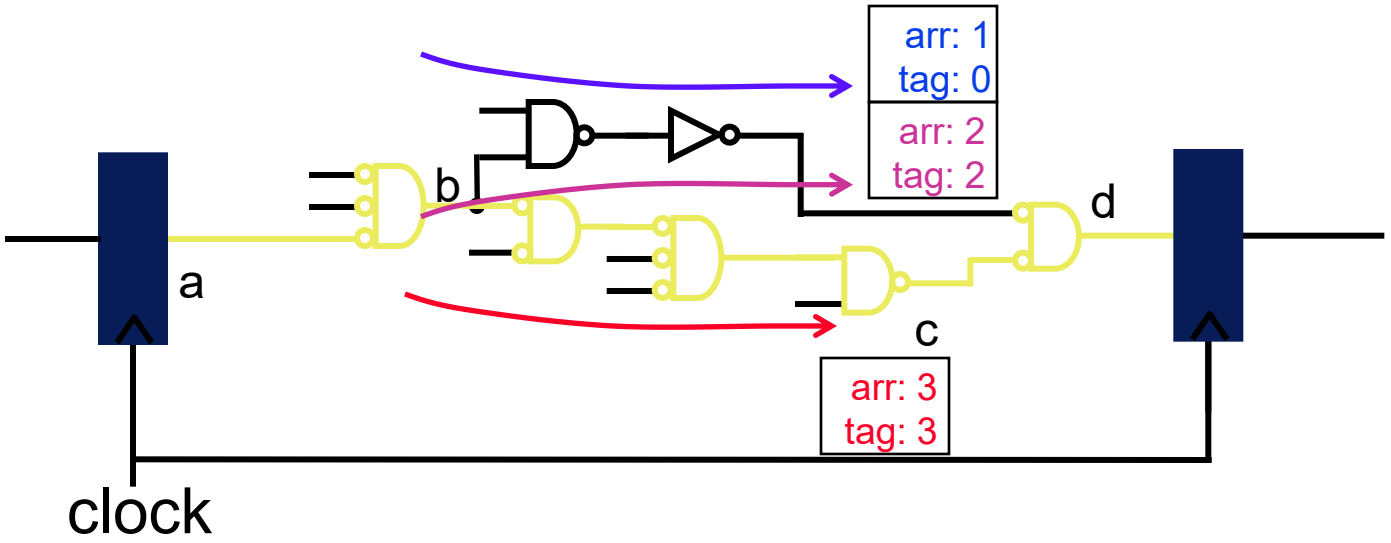
False Path Representation

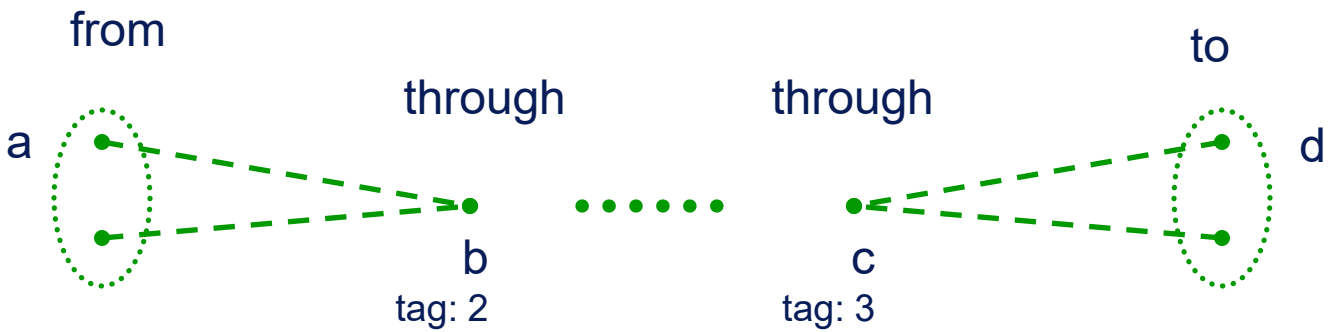
- Abstracted graph
- Set_false_path -from {…} –through {…} … -through {…} –to {…}



False Path Identification

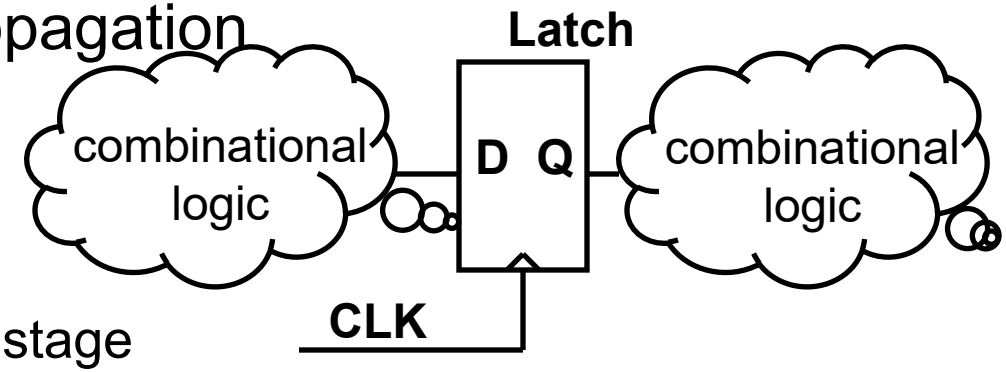
- *Tagged* timing analysis
  - Arrival times with the same tag are compared to find worst case
  - False path filtered



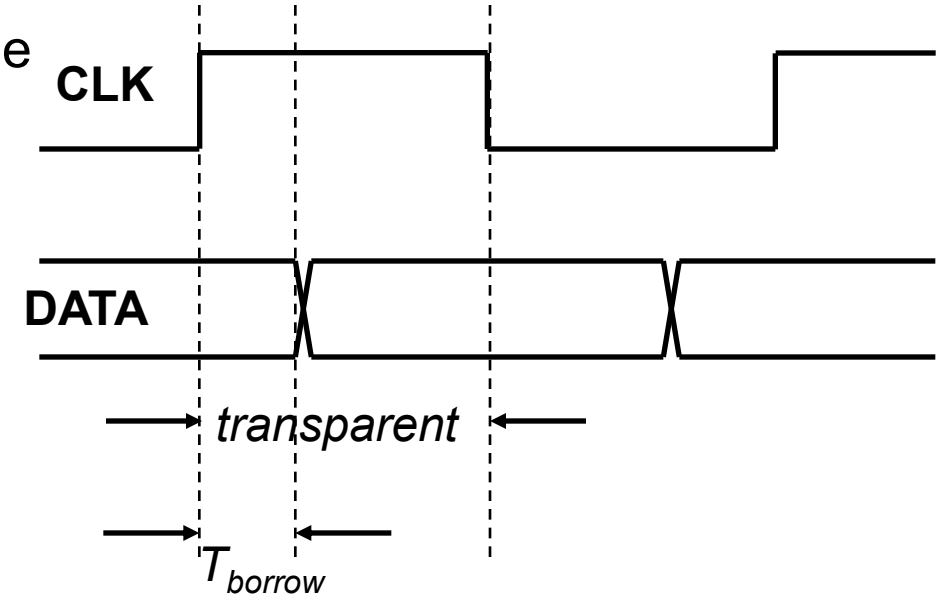


Handling Latch-Based Designs

- Latch: level enabling sequential element
- Transparent signal propagation

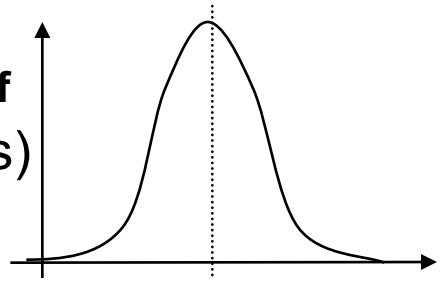


- Time borrowing
  - Path delay of previous stage – T_{borrow}
  - Path delay of current stage + T_{borrow}



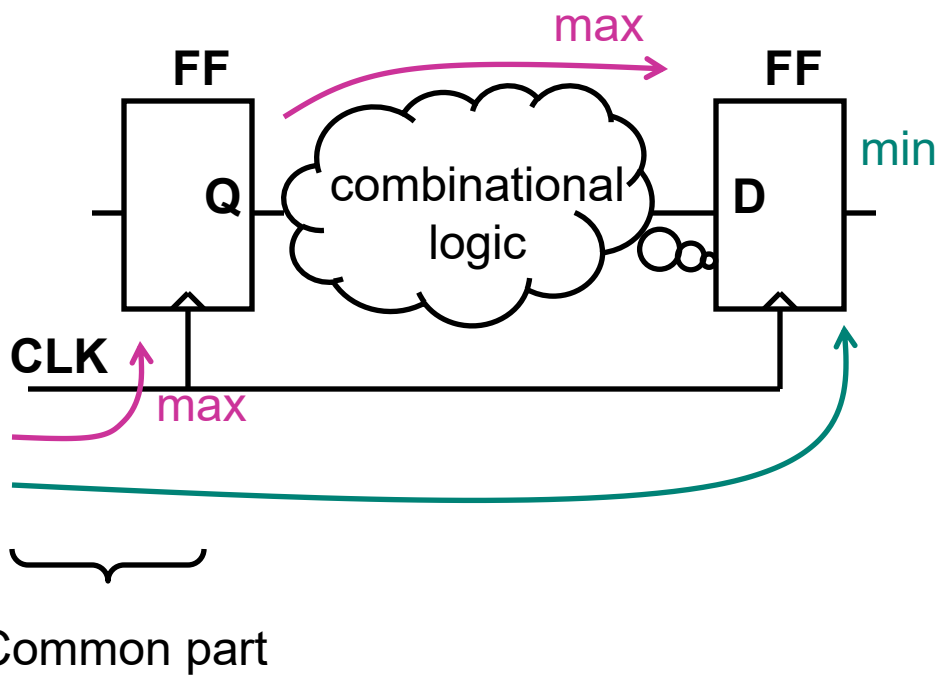
Counting Process Variation

- Off-chip variation: two paths on a chip cannot use two different operating conditions (i.e., corners) at the same time for setup or hold analysis
  - Launchclock_latepath (max) + data_latepath (max) < captureclock_earlypath (max) + clock_period – setup
  - Launchclock_earlypath (min) + data_earlypath (min) > captureclock_latepath (min) + hold
- On-chip variation: the software calculates the delay for one path based on maximum operating condition while calculating the delay for another path based on minimum operating condition for setup or hold checks
- Statistical static timing analysis (SSTA)
  - Continuous pdf (probability distribution functions)
  - Or discrete corners



Clock Re-convergence Pessimism Removal

- Common part of two clock propagation paths cannot have two different path delays at the same time
- Need to compute clock propagation delay from the branch point



Outline

- Timing Analysis
  - Timing Requirements
  - Static Timing Analysis
- Timing Correction

Timing Correction

- Driven by STA
  - “Incremental performance analysis backplane”
- Two goals
  - Fix logic design rule violations
  - Fix timing problems

Logic Design Rules

- Constraints of
  - Fanout
  - Slew rate
  - Load cap
- Reduce timing look-up table extrapolation error
- Control signal integrity
  - Transition degradation
  - Crosstalk noise
  - Supply voltage drop
  - Device reliability
- Approaches
  - Resizing
  - Buffering
  - Cloning (copying cells)

Timing Correction Approaches

- Re-synthesis
  - Local synthesis transforms
- Timing-driven placement
  - Critical net weighting
- Timing-driven routing
  - Net ordering
  - Buffering
  - Topology optimization
- Post-route optimization (IPO)
  - Re-routing
  - Re-timing and useful clock skew
  - Sizing
  - Buffering

Local Synthesis Transforms

- Resize cells
- Buffer or clone to reduce load on critical nets
- Decompose large cells
- Swap connections on commutative pins or among equivalent nets
- Move critical signals forward
- Pad early paths
- Area recovery

Transform Example

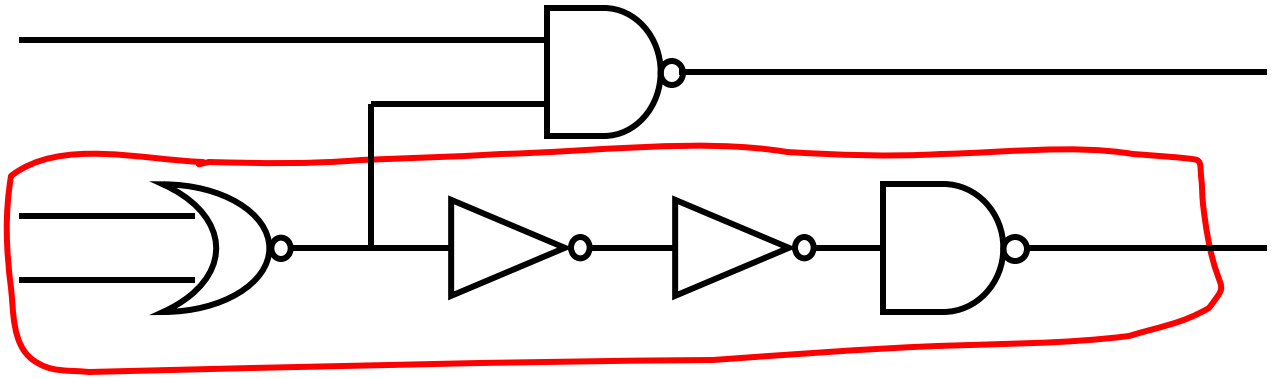
.....

Double Inverter

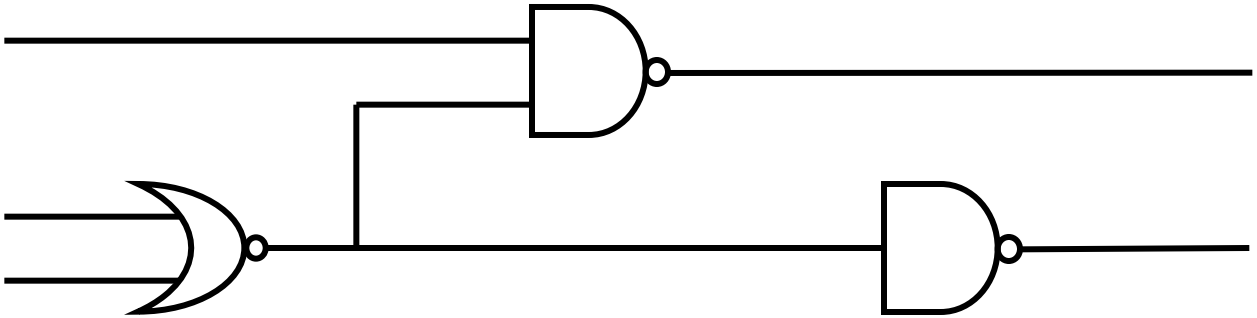
Removal

.....

.....

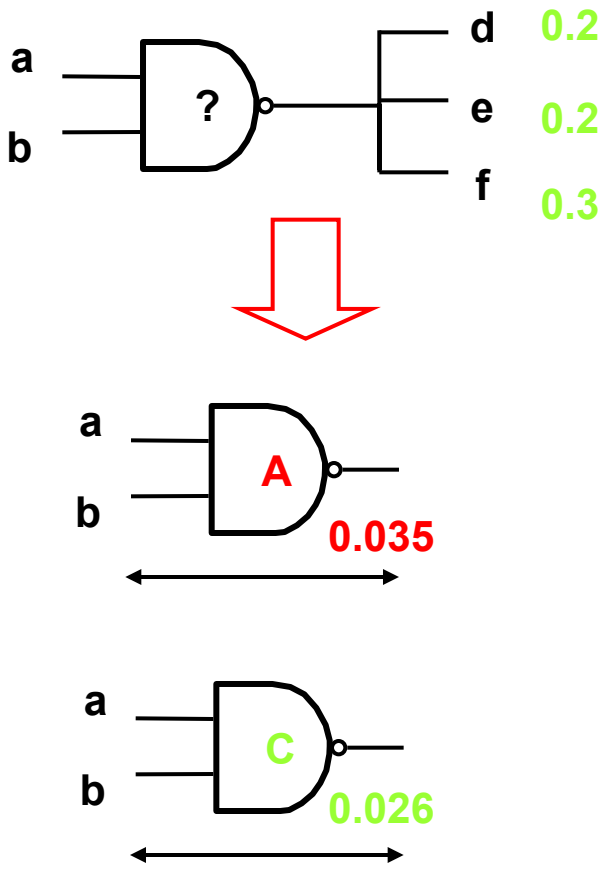


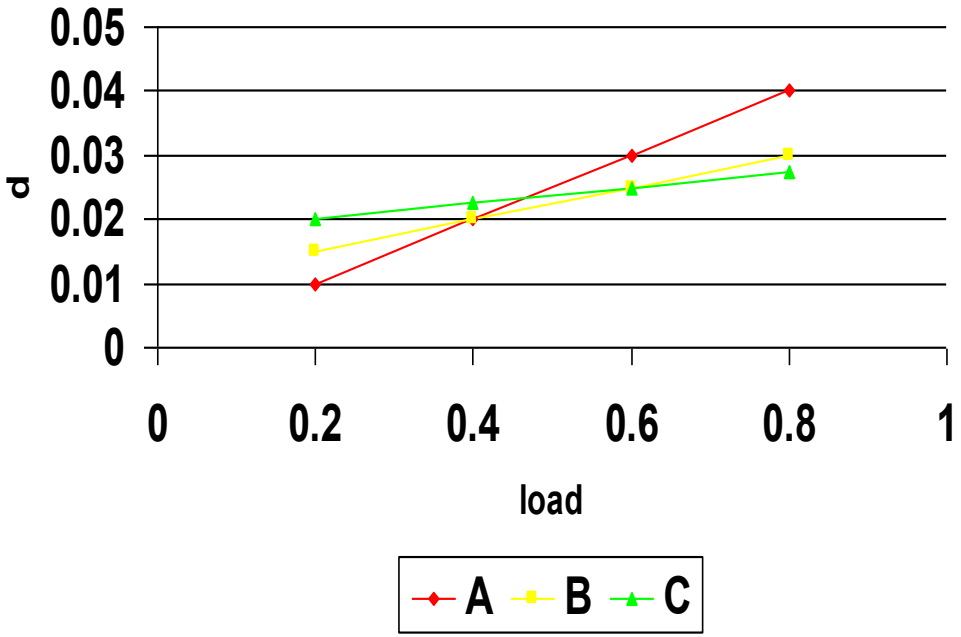
Delay = 4



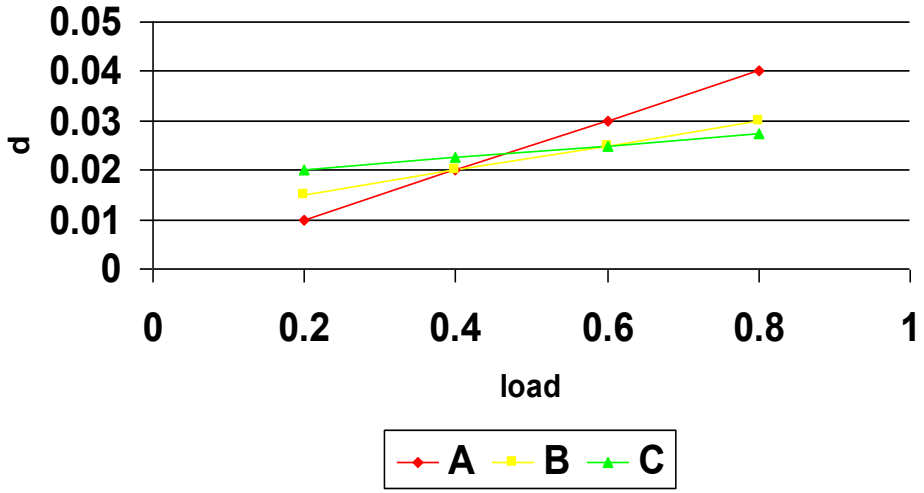
Delay = 2

Resizing


a
b
?
d
e
f
0.2
0.2
0.3
a
b
A
0.035
a
b
C
0.026

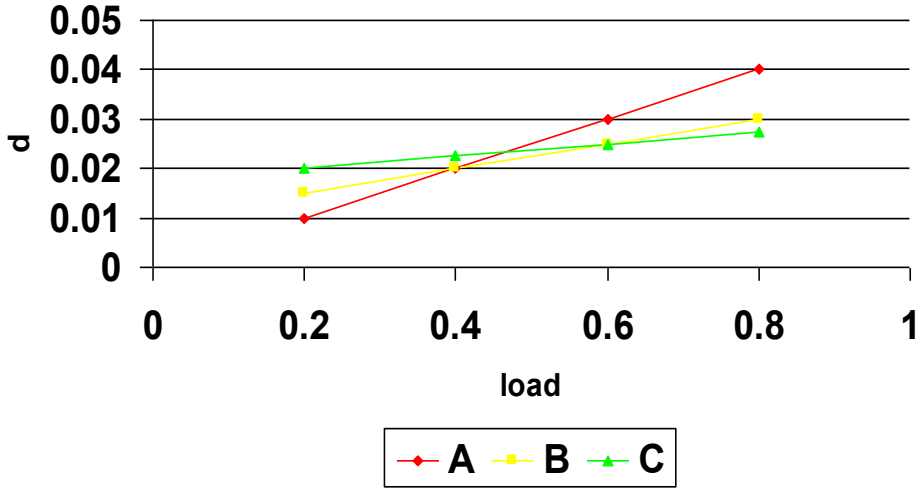

0.05
0.04
0.03
0.02
0.01
0
d
0
0.2
0.4
0.6
0.8
1
load
A
B
C

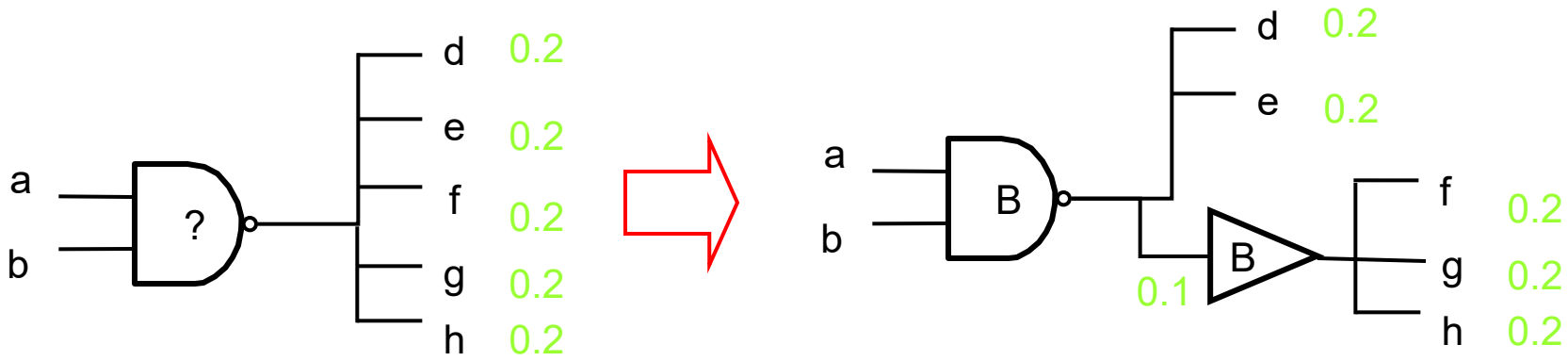
Cloning


0.05
0.04
0.03
0.02
0.01
0
d
0
0.2
0.4
0.6
0.8
1
load
A
B
C

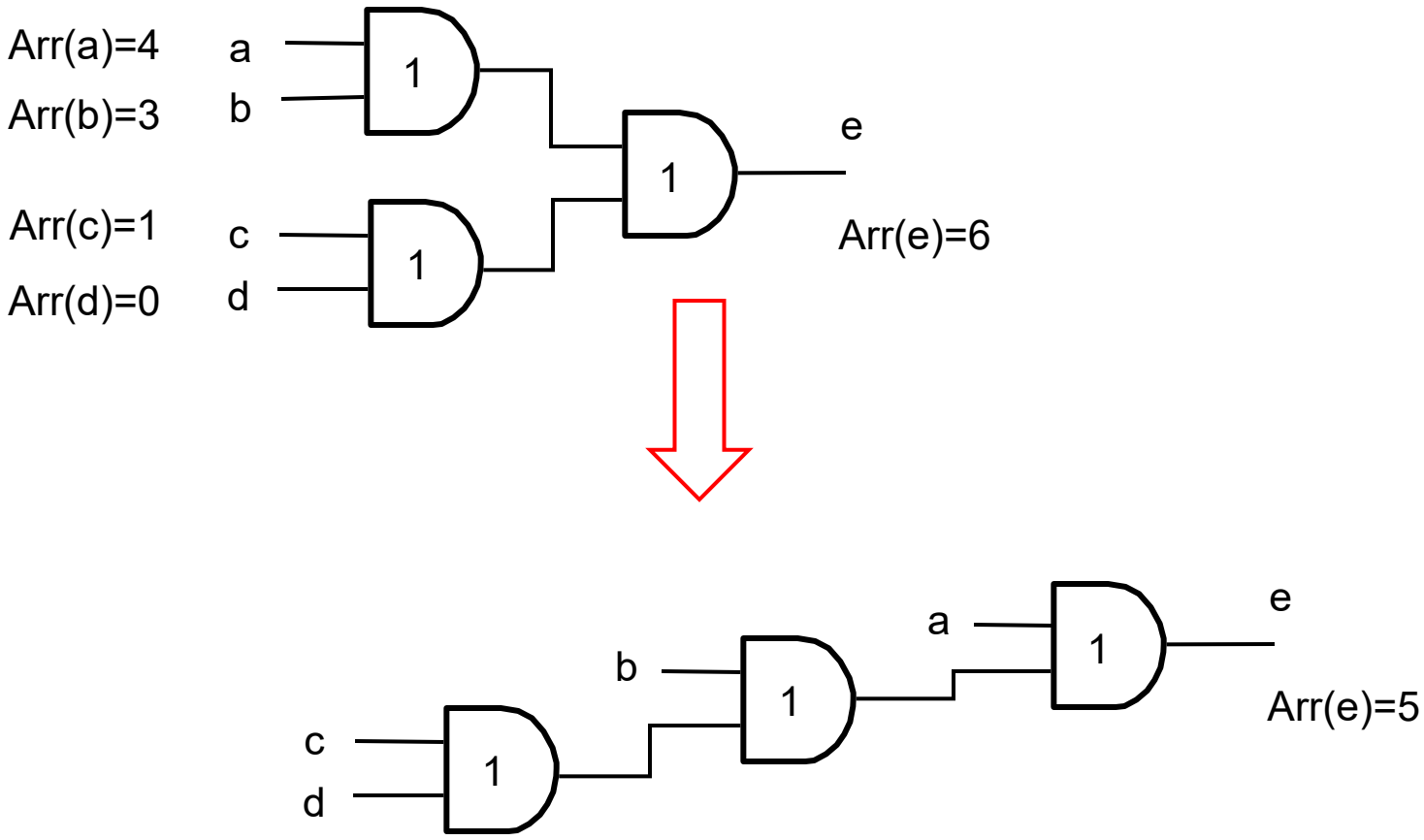

a
b
?
d
e
f
g
h
0.2
0.2
0.2
0.2
0.2
A
B
a
b
d
e
f
g
h

Buffering

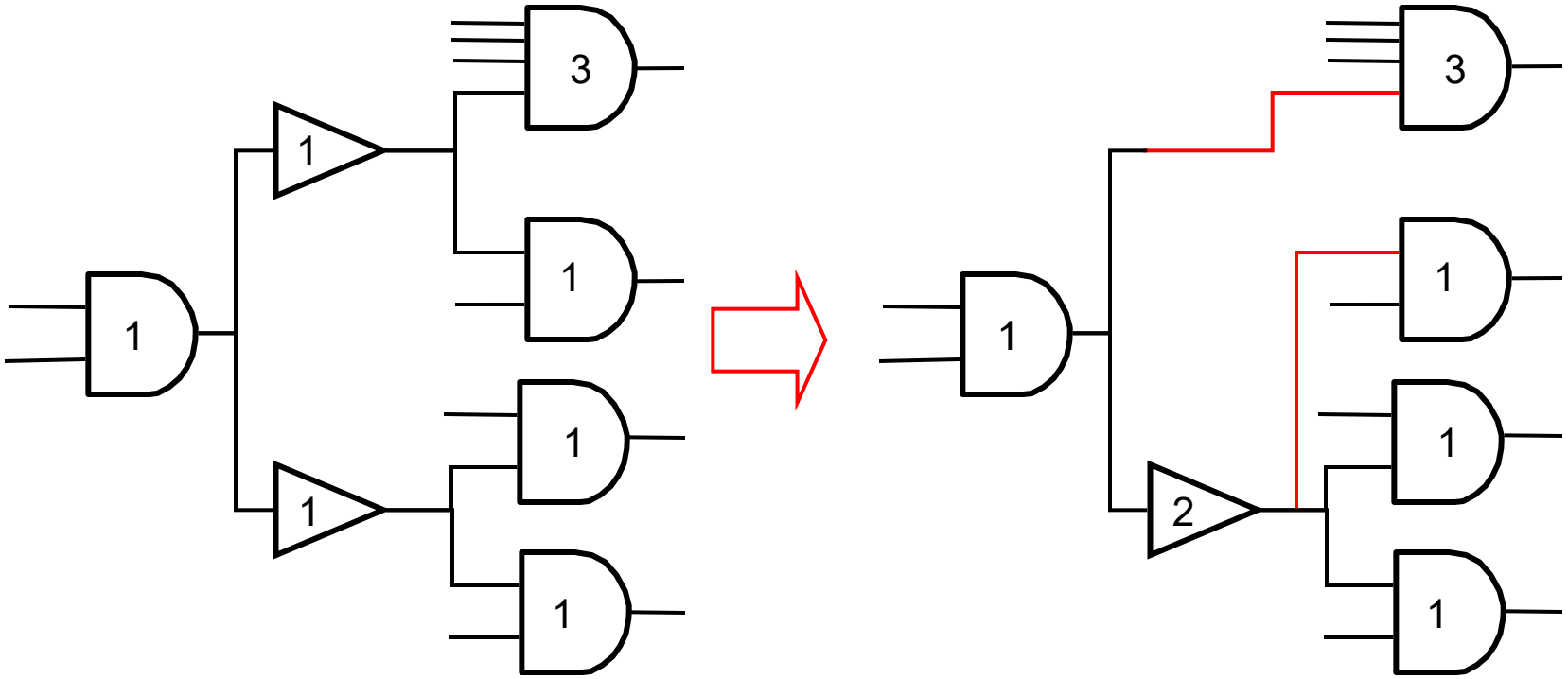

0.05
0.04
0.03
0.02
0.01
0
d
0
0.2
0.4
0.6
0.8
1
load
A
B
C


a
b
?
d
e
f
g
h
0.2
0.2
0.2
0.2
0.2
a
b
B
B
d
e
f
g
h
0.2
0.2
0.2
0.2
0.2
0.1

Redesign Fan-in Tree



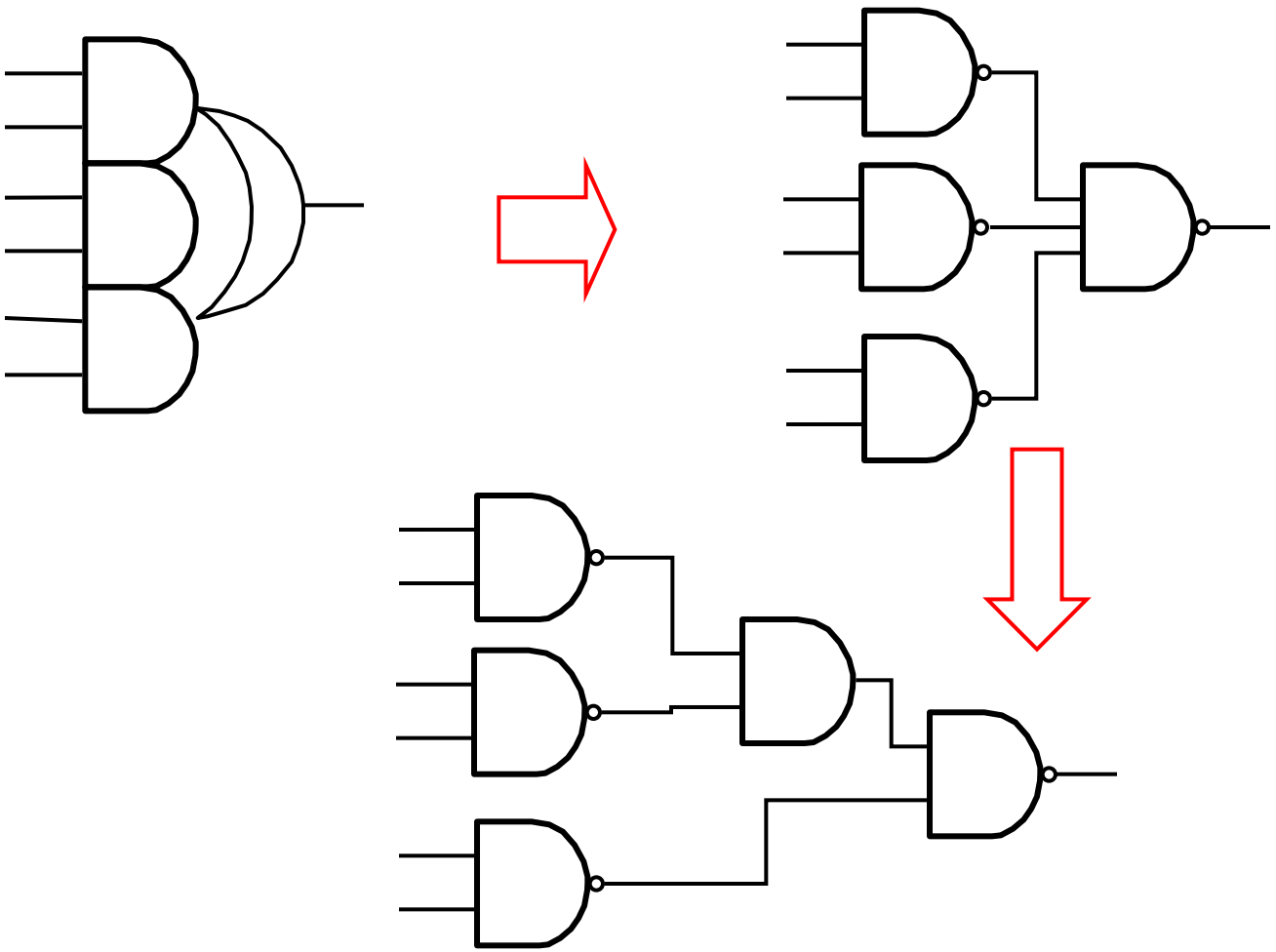
Redesign Fan-out Tree



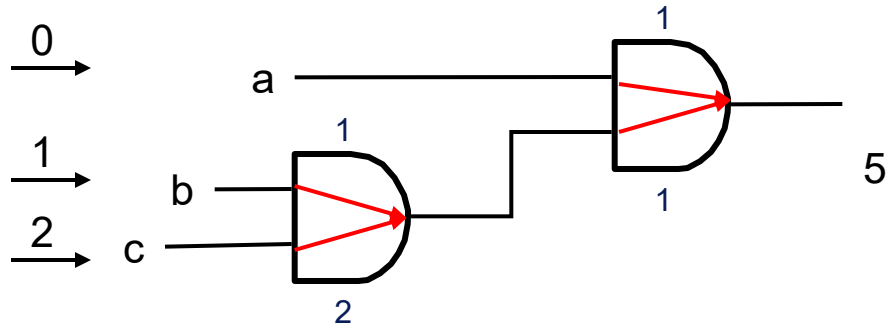
Longest Path = 5

Longest Path = 4
Slowdown of buffer due to load

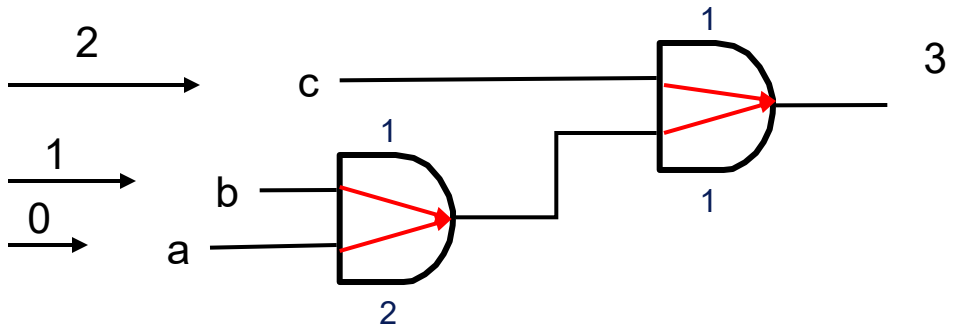
Decomposition



Swap Commutative Pins

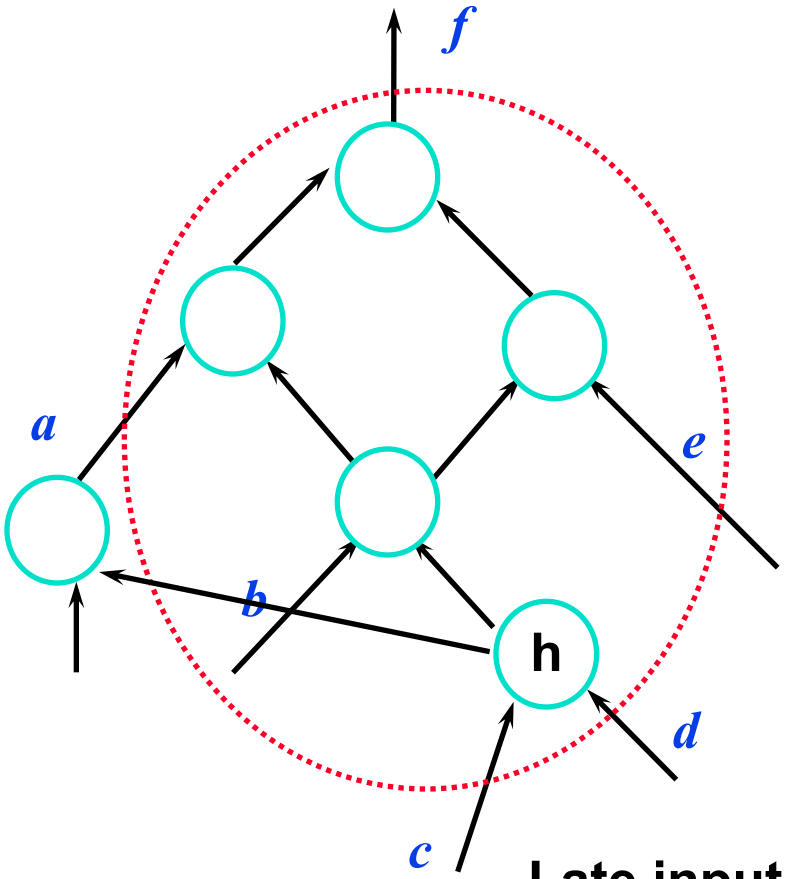


Simple sorting on arrival times and delay works

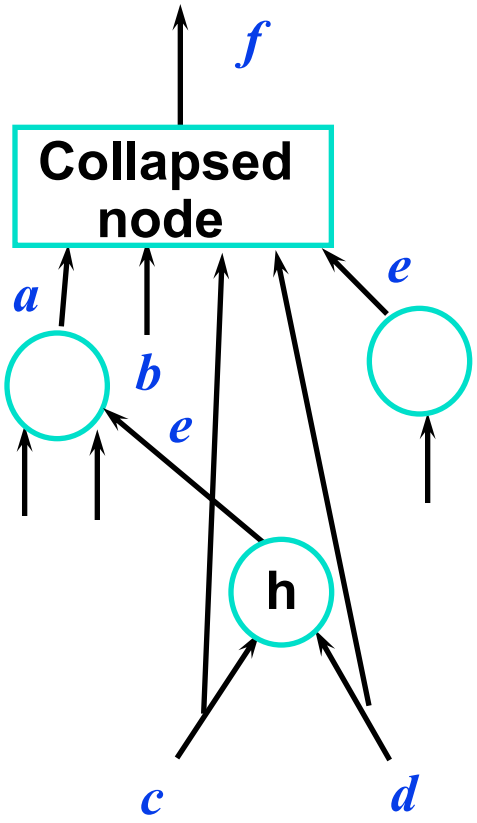


Logic Restructuring 1

- **Nodes in critical section that fan out outside of critical section are duplicated**



Late input signals

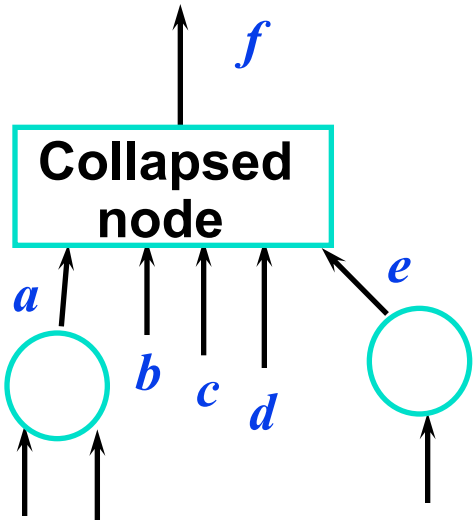


Slides courtesy of Keutzer

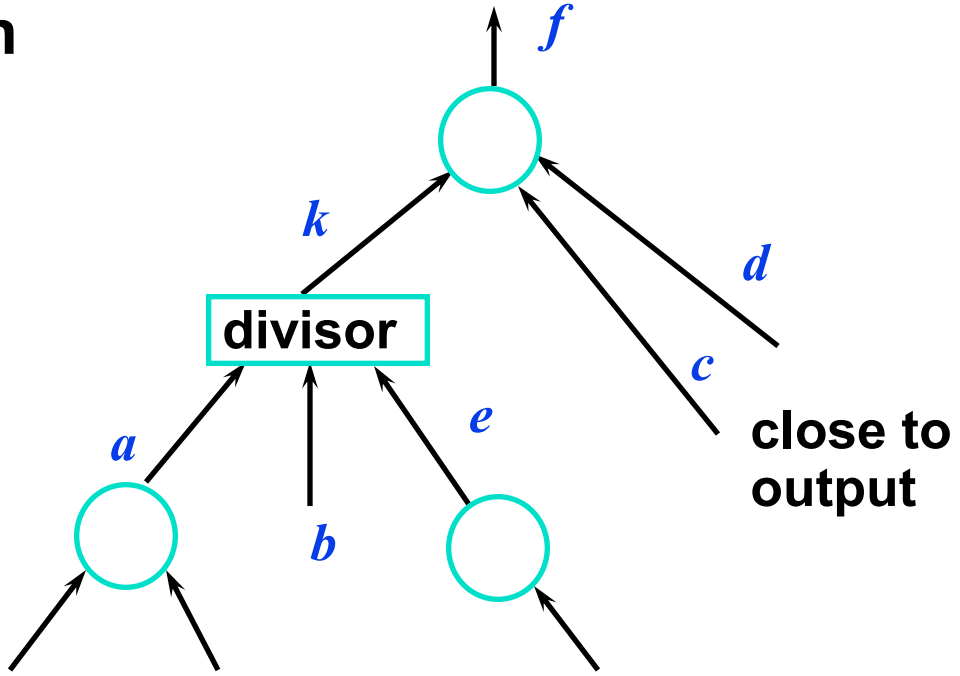
Logic Restructuring 2

- Place timing-critical nodes closer to output
  - Make them pass through fewer gates
  - After collapse, a divisor is selected such that substituting *k* into *f* places critical signal *c* and *d* closer to output

Collapse critical section



Re-extract factor *k*



Slides courtesy of Keutzer

Summary of Local Synthesis Transforms

- Variety of methods for delay optimization
  - No single technique dominates
- The one with more tricks wins? No!
- Technology dependant (for gate delay)
  - Differ with cell libraries
- Methodology dependant (for wire delay)
  - Need to predict placement and routing result
  - Uncertainty!
- Pros: large potential improvement
- Cons: less predictable, more expensive

Summary of Local Synthesis Transforms

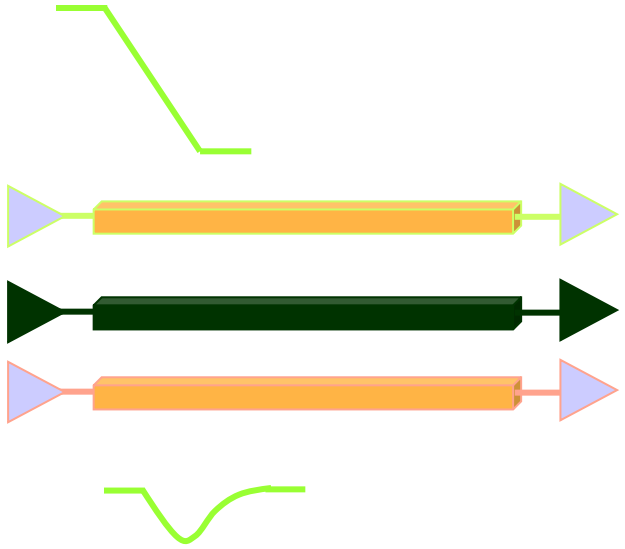
- Work smoothly in a physical synthesis flow
  - Tight integration with placement and routing
- Need a good framework for evaluating and processing different transforms
  - Accurate, fast timing engine with incremental analysis capability
    - don’t want to retime the whole design for each local transform
  - Simultaneous min and max delay analysis
    - How does fixing the setup violation affect the existing hold checks?

Timing Correction Approaches

- Re-Synthesis
  - Local Transformation
- Timing-Driven Placement
- Timing-Driven Routing
- Post-Route Optimization (IPO)
  - Re-Routing
  - Re-Timing and Useful Clock Skew
  - Sizing
  - Buffering

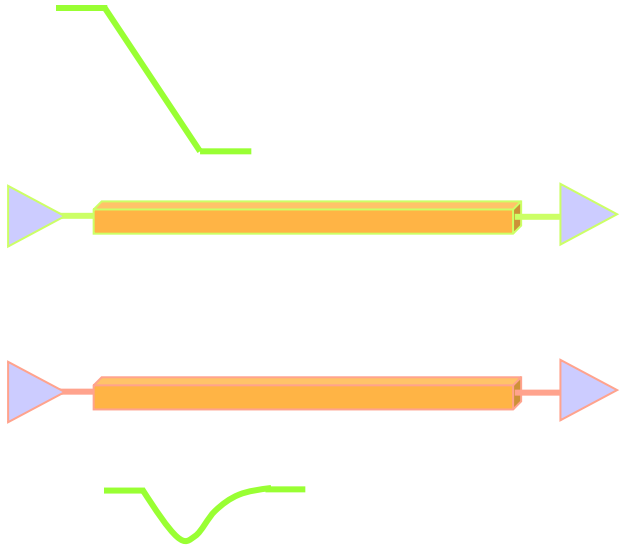
Reducing Crosstalk Effect

- Shielding
  - Effective for short range capacitive coupling
  - Not for long range inductive coupling
- Net ordering (wire swizzling)



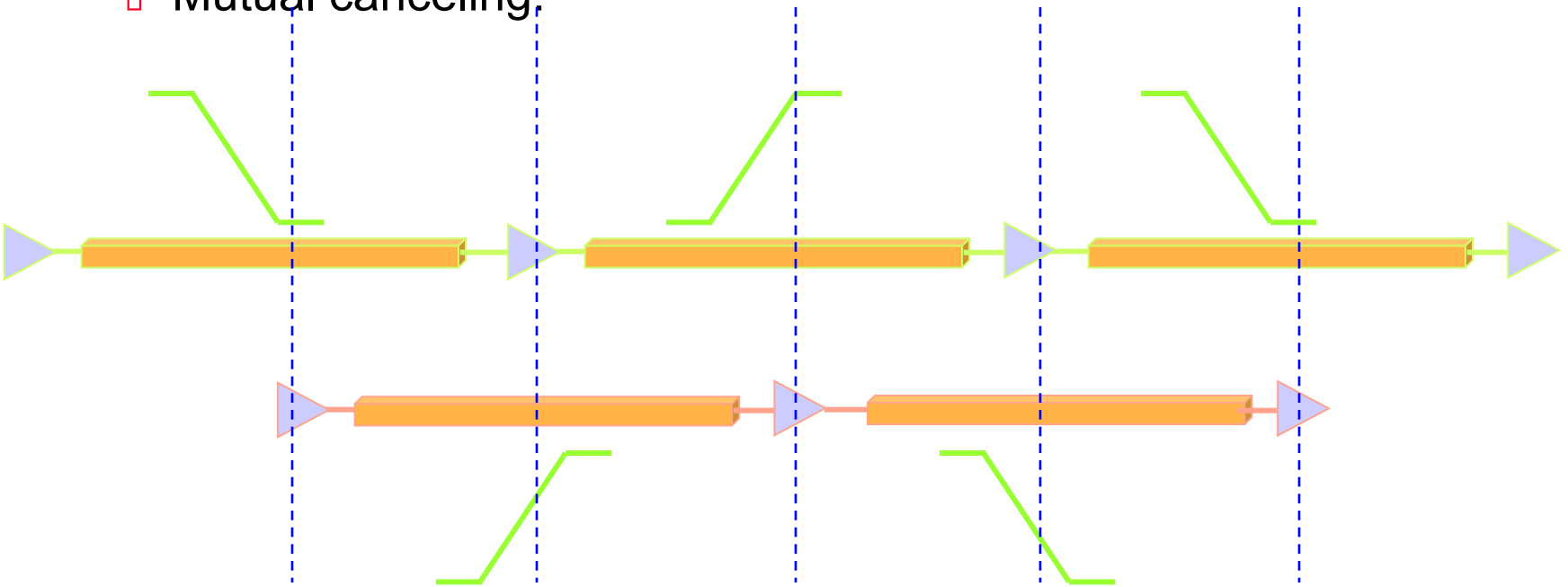
Reducing Crosstalk Effect

- Shielding
- Net ordering
- Gate sizing
  - A strong driver is less sensitive to crosstalk
  - But more likely to project crosstalk to its neighbors



Reducing Crosstalk Effect

- Shielding
- Net ordering
- Gate sizing
- Buffering
  - Partition interconnects
  - Mutual canceling:

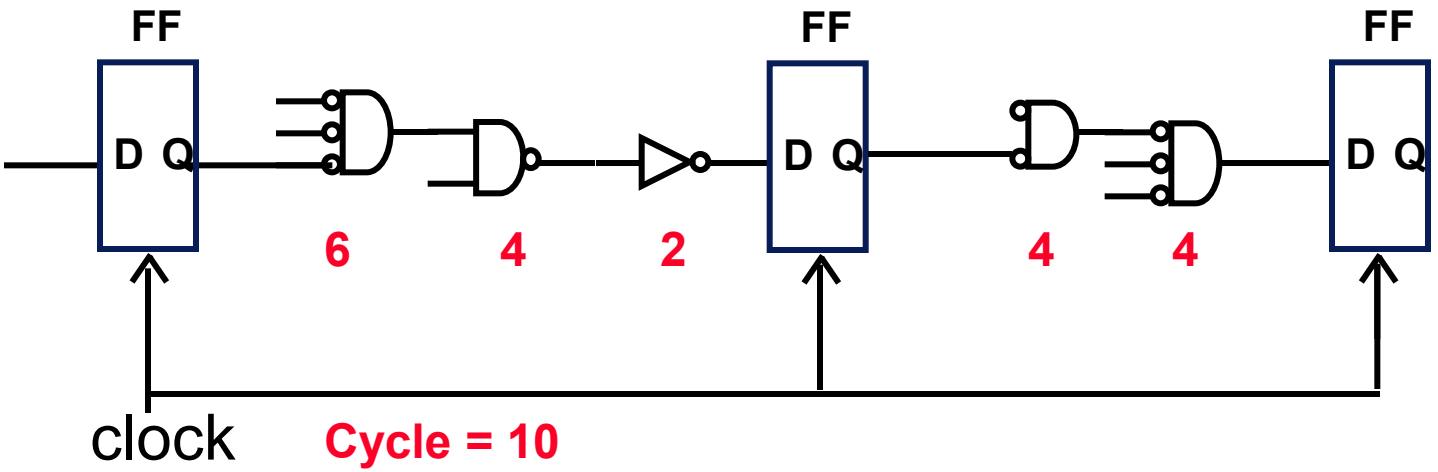


Timing Correction Approaches

- Re-Synthesis
  - Local Transformation
- Timing-Driven Placement
- Timing-Driven Routing
- Post-Route Optimization (IPO)
  - Re-Routing
  - Re-Timing and Useful Clock Skew
  - Sizing
  - Buffering

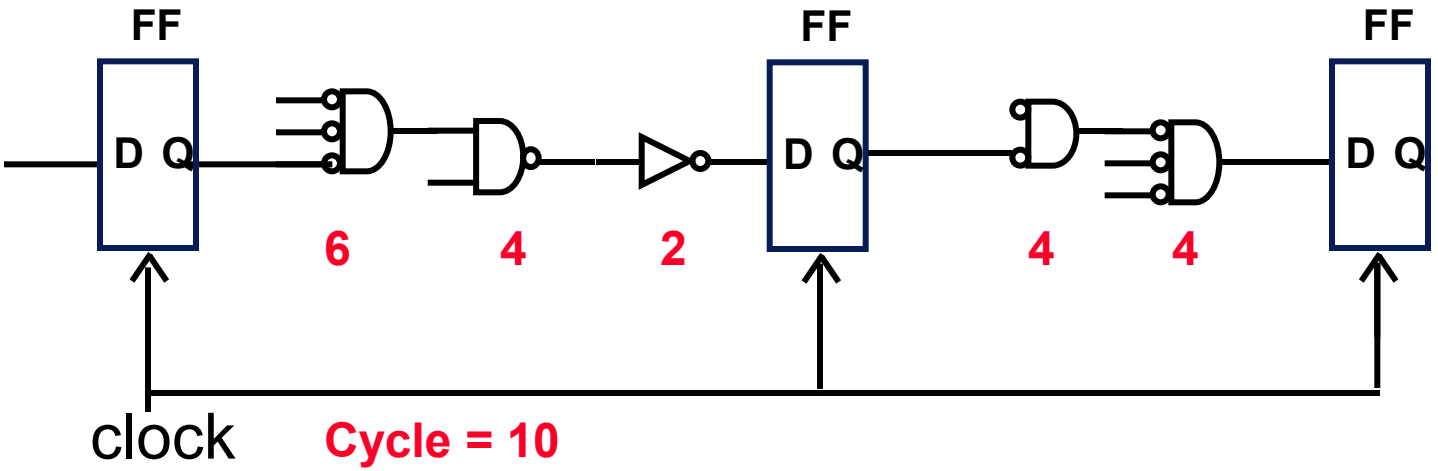
Re-Timing

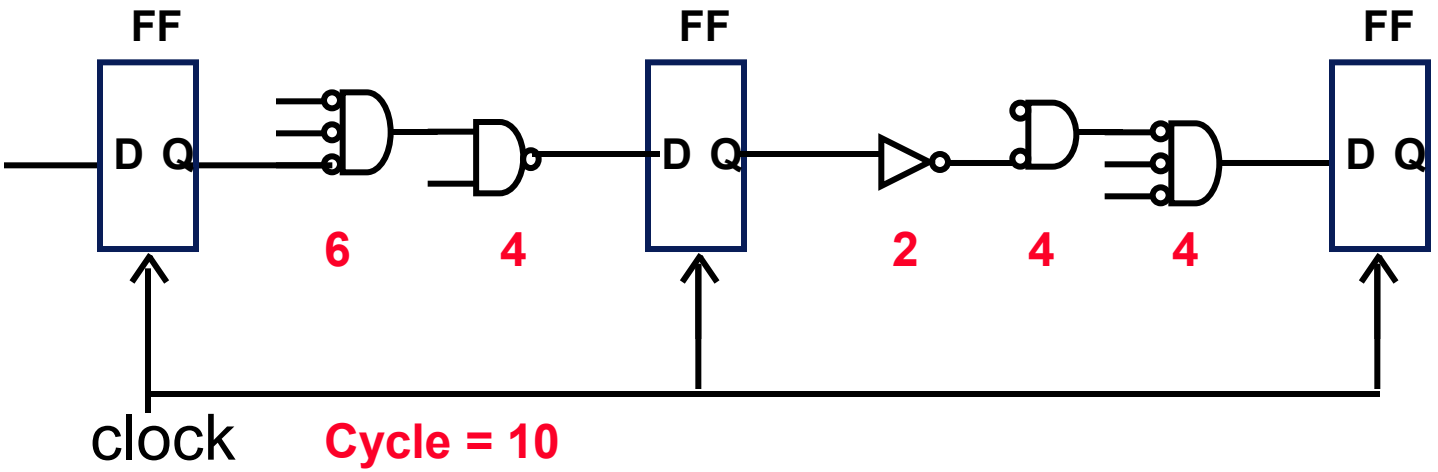
- *How would you meet the 10ns clock cycle time?*



Re-Timing

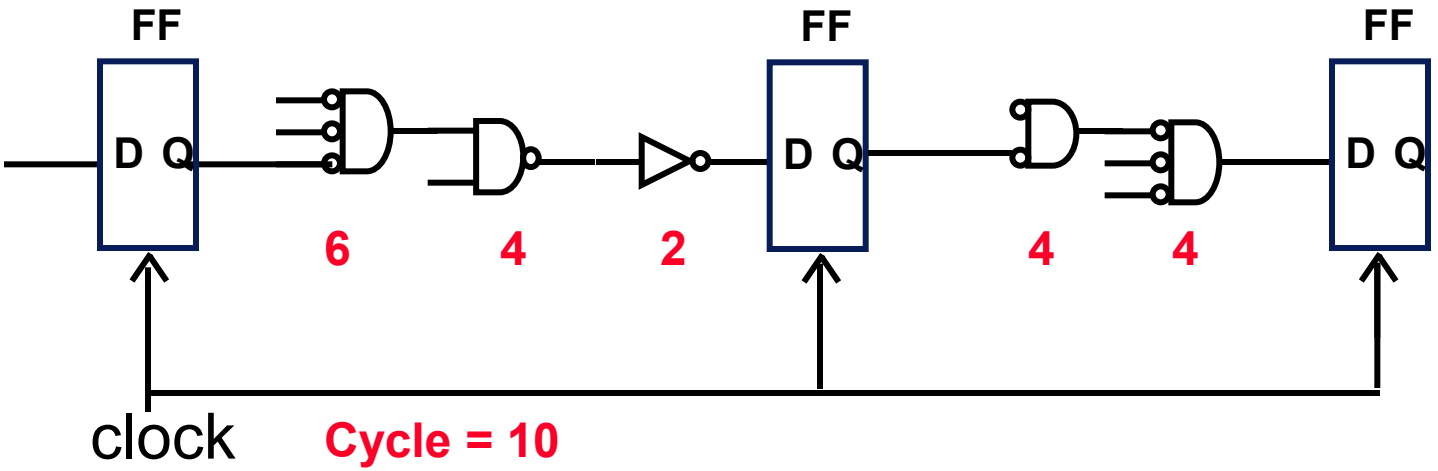
- Re-order sequential elements and combinational logic
- *Did you see a problem here?*

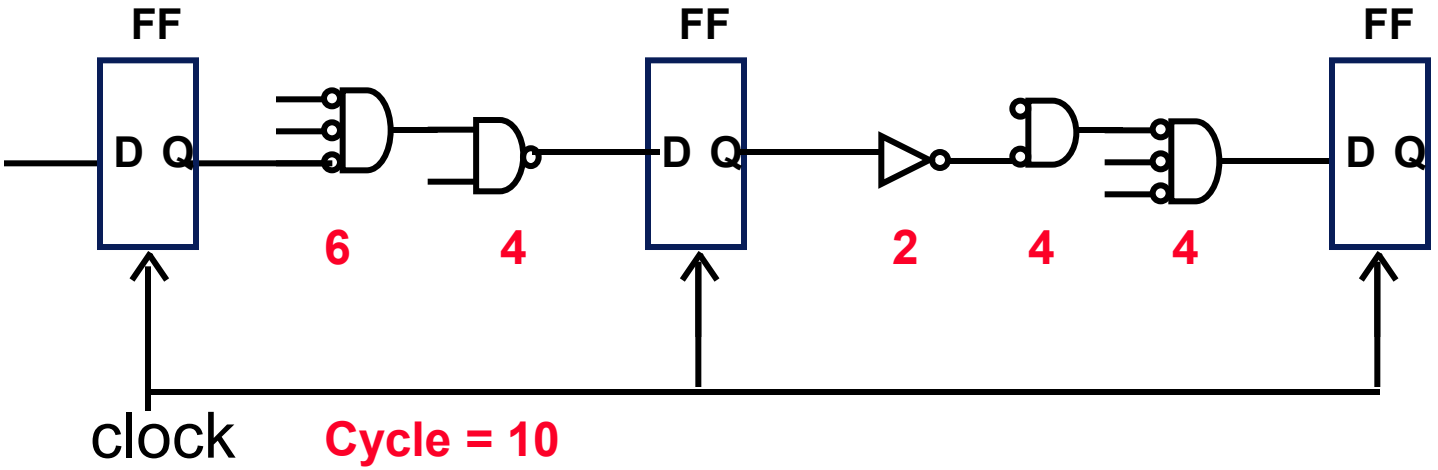




Re-Timing

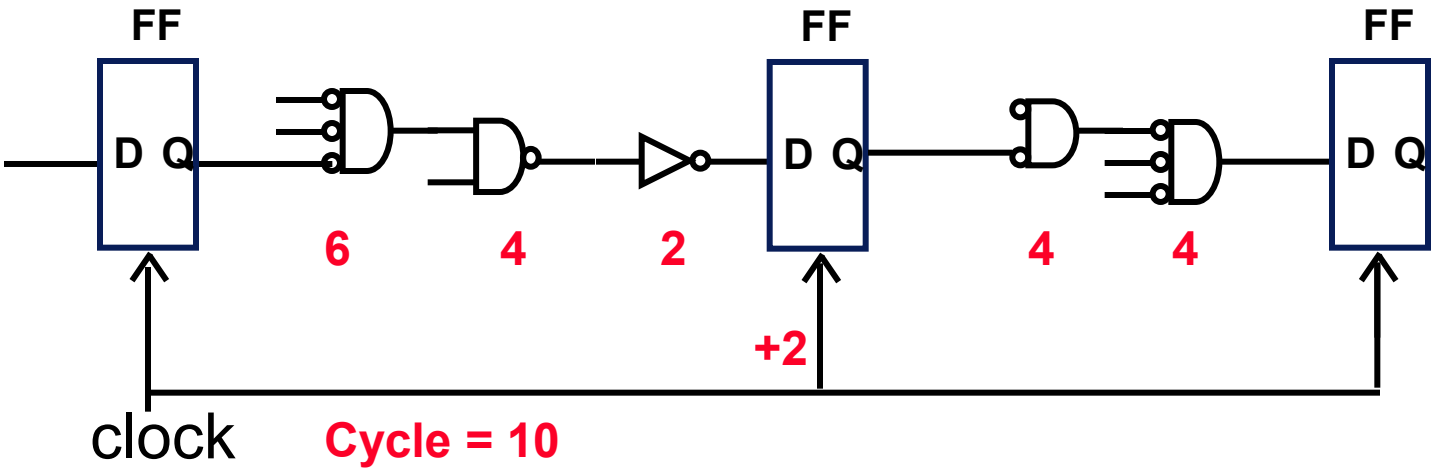
- Re-order sequential elements and combinational logic
- Need to predict placement and routing





Useful Clock Skew

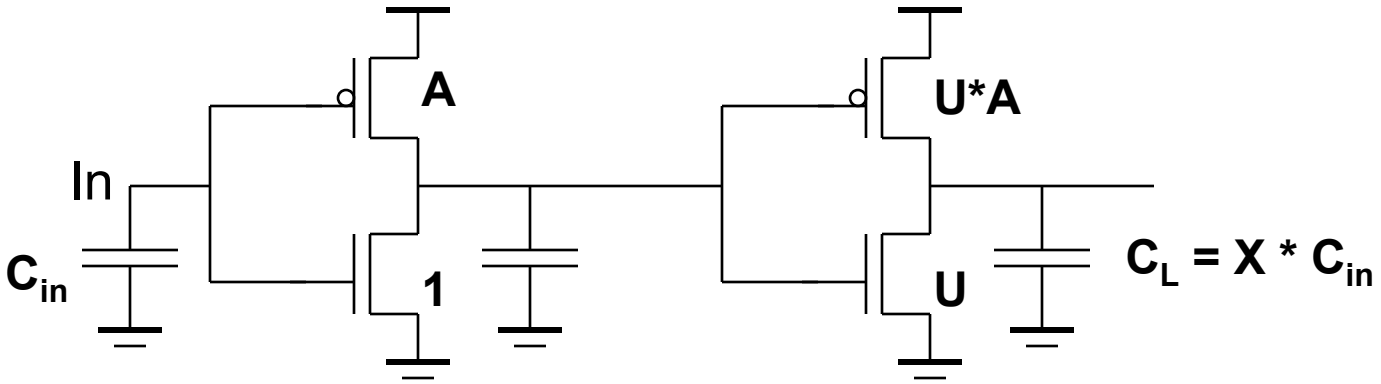
- Equivalent to re-timing
- Clock tree re-construction
  - Insert delay cells
  - Snaking
  - Add dummy capacitive load



Timing Correction Approaches

- Re-Synthesis
  - Local Transformation
- Timing-Driven Placement
- Timing-Driven Routing
- Post-Route Optimization (IPO)
  - Re-Routing
  - Re-Timing and Useful Clock Skew
  - Sizing
  - Buffering

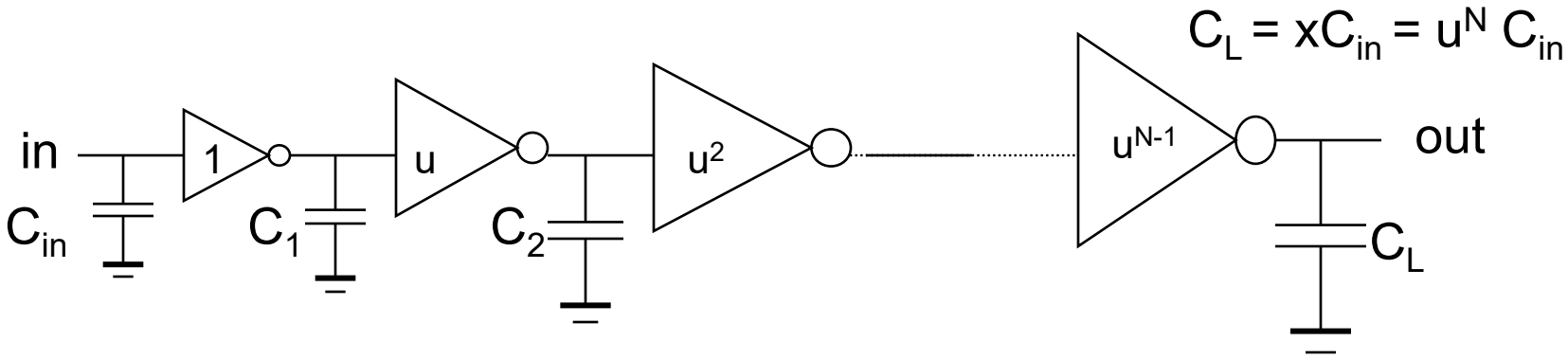
Driving Large Capacitances: Inverter As Buffer



- Total propagation delay = t_p(inv) + t_p(buffer)
- t_{p0} = delay of min-size inverter with single min-size inverter as fanout load
- Minimize $t_p = U * t_{p0} + X/U * t_{p0}$
  - U_{opt} = sqrt(X) ; $t_{p,opt} = 2\ t_{p0} * sqrt(X)$
- Use only if combined delay is less than unbuffered case

•Slide courtesy of Mary Jane Irwin, PSU

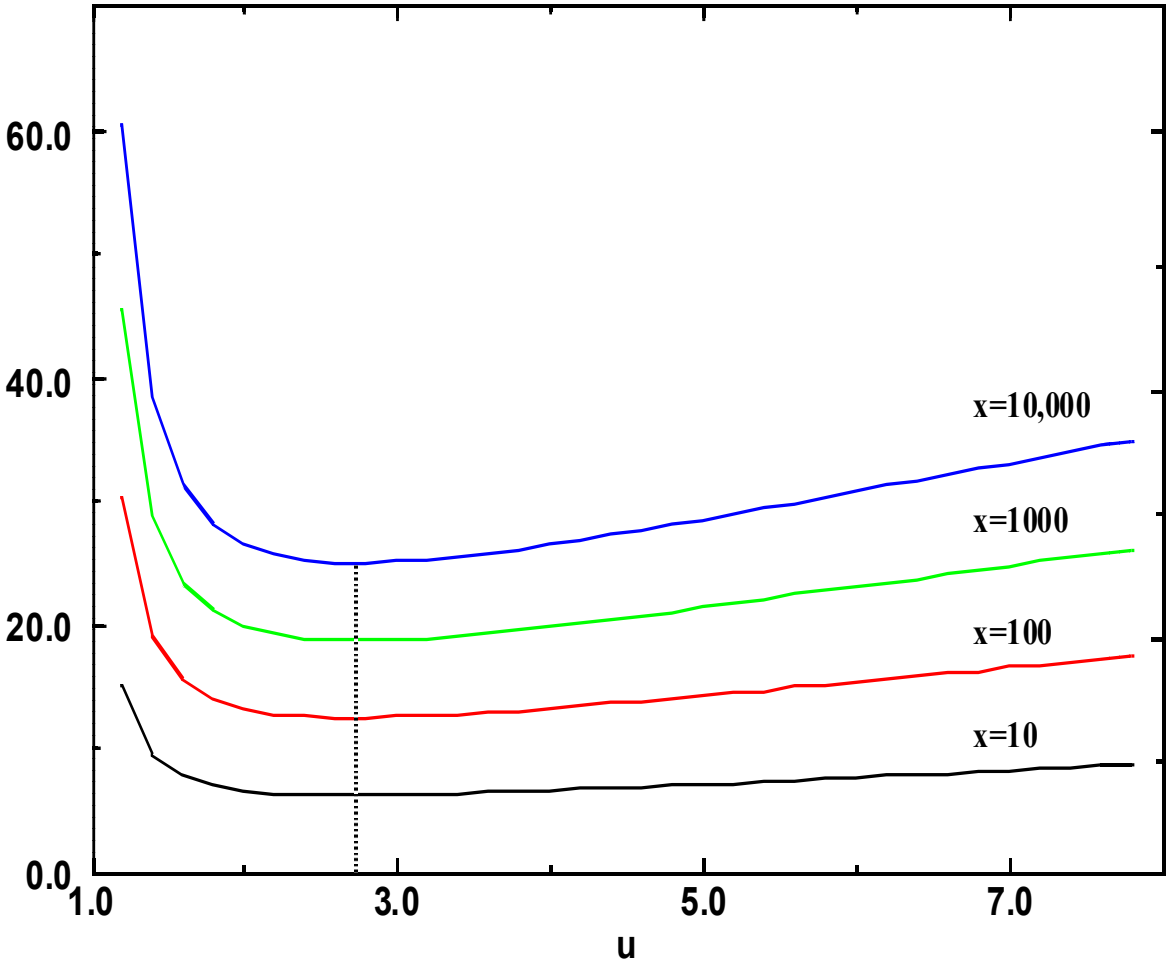
Delay Reduction With Cascaded Buffers



- Cascade of buffers with increasing sizes (U = tapering factor) can reduce delay
- If load is driven by a large transistor (which is driven by a smaller transistor) then its turn-on time dominates overall delay
- Each buffer charges the input capacitance of the next buffer in the chain and speeds up charging, reducing total delay
- Cascaded buffers are useful when $R_{int} < R_{tr}$

•Slide courtesy of Mary Jane Irwin, PSU

t_p as Function of U and X

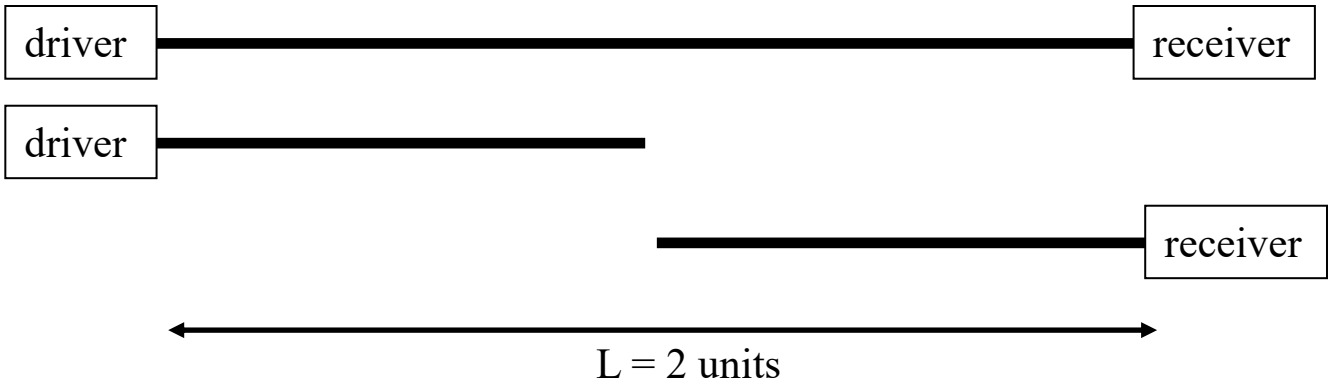


- Total line delay as function of driver size, load capacitance
- Question: Derive the optimum (min-delay) value of U.

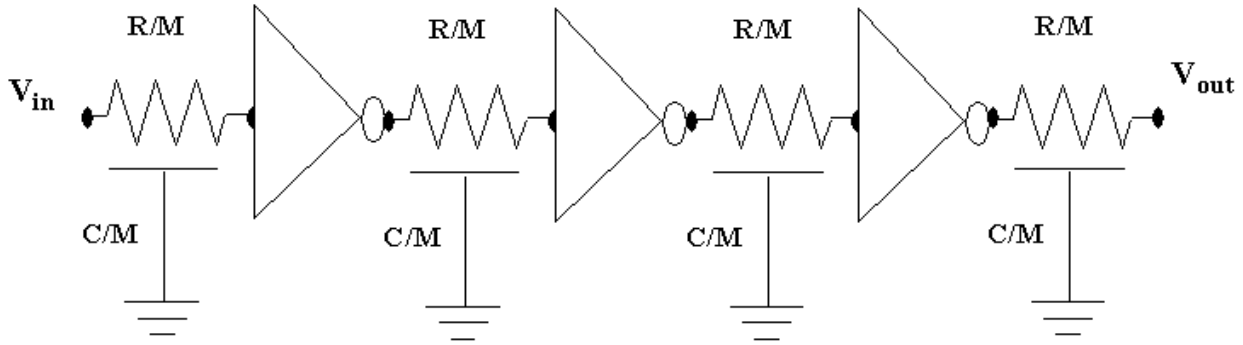
•Slide courtesy of Mary Jane Irwin, PSU

Reducing RC Delay With Repeaters

- RC delay is quadratic in length → must reduce length
- Observation: $2^2 = 4$ and $1+1 = 2$ but $1^2 + 1^2 = 2$

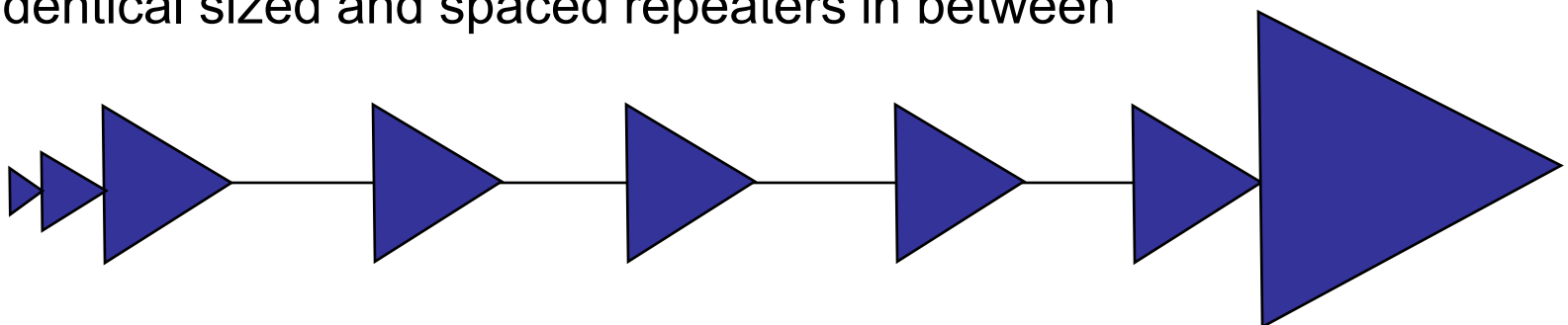


- Repeater = strong driver (usually inverter or pair of inverters for non-inversion) that is placed along a long RC line to "break up" the line and reduce delay



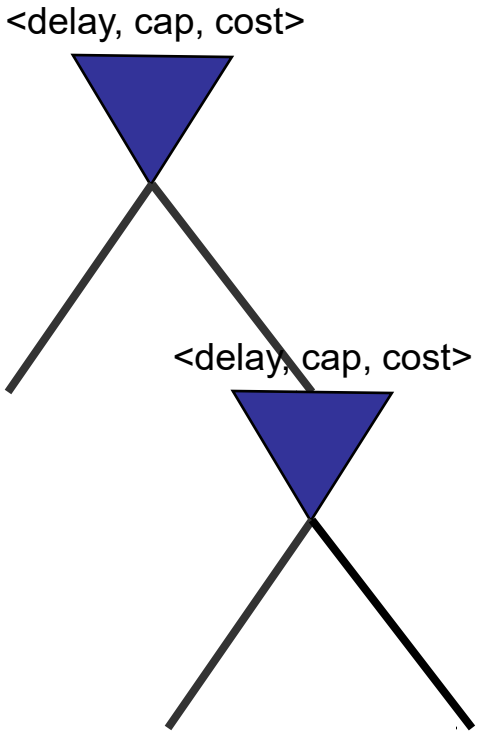
Repeaters vs. Cascaded Buffers

- *Repeaters are used to drive long RC lines*
  - *Breaking up the quadratic dependence of delay on line length is the goal*
  - *Typically sized identically*
- *Cascaded buffers are used to drive large capacitive loads, where there is no parasitic resistance*
  - *We put all buffers at the beginning of the load*
  - *This would be pointless for a long RC wire since the wire RC delay would be unaffected and would dominate the total delay*
- Optimum buffering for an uniform long interconnect
  - Cascaded buffers at source and sink
  - Identical sized and spaced repeaters in between



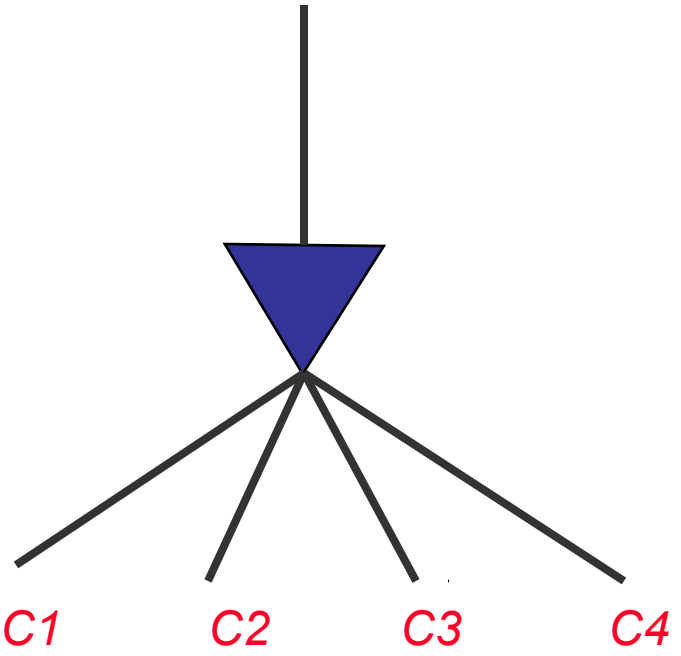
Buffering a Tree for Timing Optimization

- *Van Ginneken's* dynamic programming
  - Bottom-up traversal
  - Evaluate each sub-tree by a triple <delay, cap, cost>
  - Filter out sub-optimal solutions
- Limitations
  - Buffer insertion locations (explored by edge segmenting)
  - Buffer insertion constraints (e.g., legal buffer locations)
  - Routing detour
  - Delay calculation accuracy (wire delay, slew rate, etc.)



Buffering a Tree for Load Cap Constraints

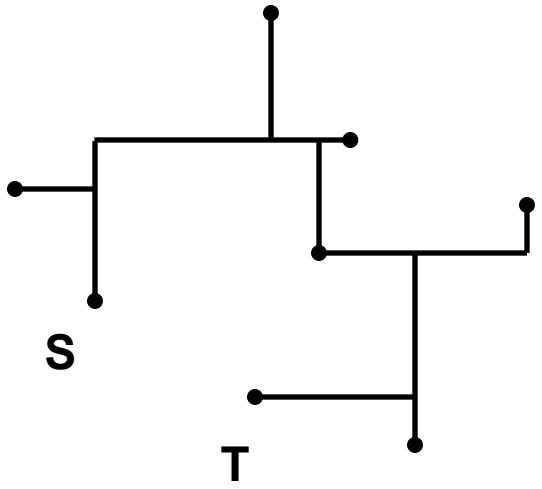
- Greedy for a single line
  - Bottom-up traversal
  - Insert a buffer when load cap reaches limit
- Greedy for a fixed routing tree
  - Bottom-up traversal
  - For each edge, greedy insertion
  - For each node, buffer the branch with the largest cap
- NP-hard for simultaneous buffering and routing construction



$C1 < U, C2 < U, C3 < U, C4 < U$
$C1 + C2 + C3 + C4 > U$

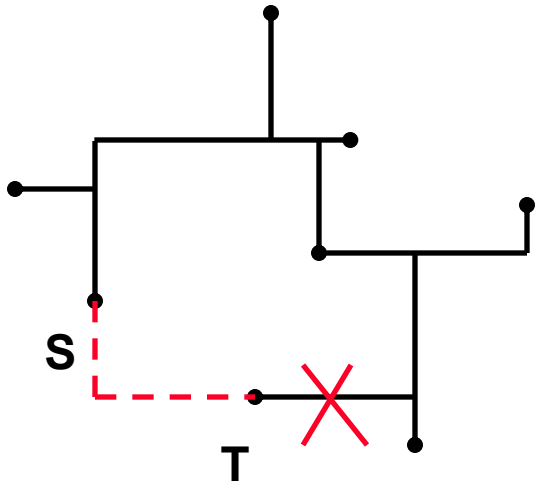
Timing-Driven Routing Tree Construction

- Minimum wirelength (Steiner Minimum Tree)
- Given a set of terminals S
- Find an additional set of points A such that a spanning tree T over S ∪ A has minimum wirelength
- May not be timing optimum
  - Some sinks are more timing critical than others
  - Some sinks have larger capacitive load
  - Buffers?



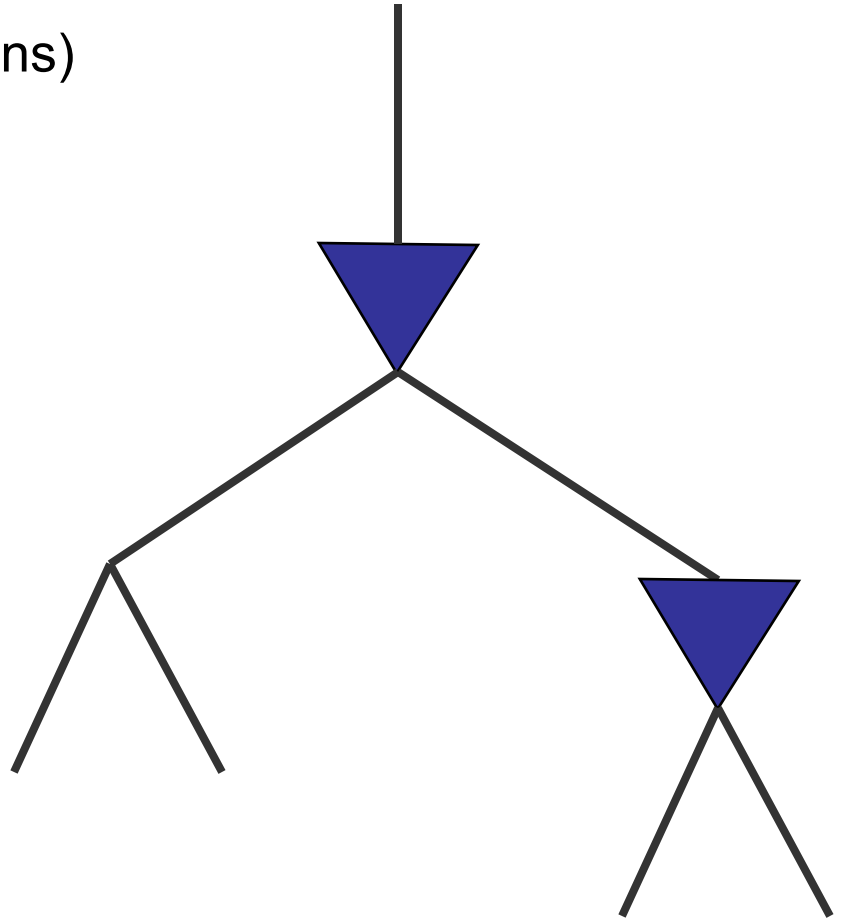
Timing-Driven Routing Tree Construction

- Minimum wirelength (Steiner Minimum Tree)
- Shortest Path Tree
- AHHK Tree
  - Cost(q) = k * path_length(p) + edge_length(p, q)
  - k = 0 ➔ minimum wirelength
  - k = 1➔ shortest path
- Heuristics with sink timing criticality weights



Timing-Driven Routing Tree Construction

- Simultaneous routing tree construction and buffer insertion
  - Buffer station (legal buffer locations)
  - Routing blockage
- Dynamic programming
  - P-Tree
- Clustering (C-Tree)
  - Timing criticality
  - Geometric distance
  - Signal polarity
  - Try AHHK with different k



Timing-Driven Routing Tree Topology Optimization

- Chicken-egg dilemma (delay vs. routing)
- Iterative greedy improvement (Q-Tree)

